

IMERSÃO CORE DEV — EDIÇÃO JAVA

Plano de estudos: aprenda a programar em Java

APRENDA A PROGRAMAR EM JAVA

PLANO DE ESTUDOS — IMERSÃO CORE DEV

+ de 200 exercícios separados em 15 lições estratégicas para destravar o seu aprendizado

Caro Leitor

Os primeiros passos no aprendizado da programação são sempre desafiadores. Seja pela falta de informação ou até mesmo pelo excesso dela. Várias linguagens, outros tantos pontos de vista e incontáveis videoaulas disponíveis na internet podem mais atrapalhar e confundir do que realmente ajudar quem está começando.

Com Java, esse excesso costuma ser ainda maior. Quem pesquisa “aprender Java” hoje encontra tutoriais de Spring Boot, microsserviços, injeção de dependência e arquitetura hexagonal — assuntos excelentes, porém completamente fora de ordem para alguém que ainda não escreveu o primeiro laço de repetição. É como tentar aprender a dirigir estudando o manual de manutenção do motor.

Por isso, se você quer iniciar na programação com Java, ou já começou e se sente inseguro e um pouco travado para dar os primeiros passos, eu quero te dizer que este livro é para você.

Nós acreditamos que para aprender programação você não precisa apenas de informações técnicas. Você precisa de um bom plano de estudos, com direcionamento, foco e objetivos.

Por esse motivo criamos este ebook com as principais aulas da nossa Imersão Core Dev, agora escritas inteiramente para quem escolheu Java como primeira linguagem.

São 15 lições e mais de 200 exercícios, entre propostos e resolvidos, que vão ajudar você a sair do zero e escrever os seus primeiros programas com segurança.

Nas primeiras duas lições, nosso objetivo é entender o raciocínio lógico e como criar um pensamento computacional. Não há código nenhum aqui, e isso é proposital: quem pula esta etapa costuma travar mais adiante.

Nas lições de 3 a 7 você vai aprender a estruturação básica de um programa Java, dominando todos os comandos necessários para a criação de pequenos programas sequenciais.

Nas lições 8 e 9 você vai aprender a trabalhar com situações que envolvem comandos de decisão, onde o programa executa tarefas diferentes dependendo da informação que receber.

Nas lições 10 e 11 você vai aprender a estruturar seu código em pequenos métodos que podem ser reutilizados, e também vai aprender a trabalhar com os métodos prontos da linguagem para realizar a manipulação de textos.

Na lição 12 você vai aprender a trabalhar com listas de informações em memória dentro dos seus programas.

Nas lições 13 e 14 você vai aprender a trabalhar com comandos de repetição de código.

Por último, na lição 15, você vai aprender a estruturar uma aplicação de cadastro completa, dividida em camadas, utilizando todos os conhecimentos adquiridos nas lições anteriores.

Uma observação importante antes de começar: **este livro é para ser praticado, não lido.** Cada lição traz exercícios resolvidos, que você deve acompanhar digitando junto, e exercícios propostos, que você deve tentar resolver antes de procurar qualquer resposta. Ler código é confortável e ensina pouco. Escrever código é desconfortável e ensina tudo.

Bons estudos!

Sumário

Lição	Tema
DIA 01	Sequência lógica
DIA 02	Programação descritiva
DIA 03	Primeiros programas
DIA 04	Como guardar informações na memória do computador
DIA 05	Trabalhando com operações matemáticas
DIA 06	Conheça os operadores compostos
DIA 07	Como fazer comparações
DIA 08	Desvio de fluxo com os comandos if/else
DIA 09	Desvio de fluxo com o comando switch/case
DIA 10	Trabalhando com métodos
DIA 11	Métodos para manipulação de texto
DIA 12	Como trabalhar com listas
DIA 13	Laços de repetição com o comando while
DIA 14	Laços de repetição com o comando for
DIA 15	Aplicação de cadastro completa

Como usar este livro

Cada lição segue sempre a mesma estrutura, e cada parte tem uma função:

Objetivo do dia — o que você deve ser capaz de fazer ao terminar a lição. Se ao final você não conseguir, releia antes de avançar.

Introdução — a conexão com o que você já aprendeu e o que vem a seguir.

Conteúdo da aula — a explicação dos conceitos, sempre acompanhada de código.

Vamos analisar o código — a leitura linha a linha do programa de exemplo. É a parte mais importante do livro. Não pule.

De olho nos detalhes — os pontos em que quase todo mundo tropeça, e como evitá-los.

Exercícios resolvidos — problemas com a solução comentada. Digite cada um deles, não copie e cole.

Exercícios — problemas para você resolver sozinho. Este é o momento em que o aprendizado acontece de verdade.

Dica do dia — uma informação para fixar antes de dormir.

Reserve entre 40 e 90 minutos por lição. Se um dia render menos, tudo bem: é melhor fazer 15 dias honestos do que 5 dias apressados.

DIA 01 — SEQUÊNCIA LÓGICA

Objetivo do dia

Desenvolver a habilidade de descrever as ações de uma tarefa de forma detalhada.

Introdução à programação

Um programa é basicamente uma sequência de comandos (ações) que são executados para realizar uma determinada tarefa.

O trabalho de um programador é escrever essa sequência informando quais comandos precisam ser executados e em qual ordem estes comandos precisam ocorrer.

Um dos erros mais comuns quando estamos criando os nossos primeiros programas é não conseguir identificar quais são os comandos necessários para realizar uma tarefa, e em que sequência estes comandos precisam ser executados. Isso acontece porque no dia a dia o nosso cérebro automatiza várias tarefas que realizamos, tirando da nossa memória consciente a visualização de cada ação de maneira individual.

Por exemplo, quando estamos nos alimentando, não ficamos o tempo todo pensando sobre cada etapa do processo: segurar o garfo, pegar a comida no prato, levar a comida à boca, abrir a boca,

colocar a comida dentro da boca, fechar a boca, remover o garfo da boca, mastigar o alimento, engolir o alimento. Esse processo acontece de forma natural para nós. No entanto, se formos programar um robô para simular o processo de se alimentar, precisaremos descrever cada uma dessas ações, uma a uma, na ordem exata.

Programar é, antes de qualquer coisa, reaprender a enxergar o óbvio.

As três características de uma boa sequência

Toda sequência bem escrita tem três características. Guarde-as, porque você vai usá-las durante todo o livro:

- 1. Os passos são atômicos.** Cada passo descreve uma única ação, que não precisa ser subdividida para ser compreendida.
- 2. Os passos estão em ordem obrigatória.** Trocar a ordem de dois passos deve quebrar a tarefa. Se não quebrar, provavelmente falta detalhamento.
- 3. Os passos são completos.** Nada fica implícito. Se você precisou “supor” alguma coisa ao ler a sequência, essa suposição é um passo que faltou.

Exemplo comentado: fazer um café

Veja a diferença entre a descrição humana e a descrição para uma máquina.

Descrição humana: “pego a água, esquento, coloco o pó e coo”.

Descrição para uma máquina:

1. Pegar a chaleira.
2. Abrir a torneira.
3. Encher a chaleira com 500 ml de água.
4. Fechar a torneira.
5. Colocar a chaleira sobre a boca do fogão.
6. Acender o fogo.
7. Aguardar até a água ferver.
8. Apagar o fogo.
9. Posicionar o coador sobre a garrafa.
10. Colocar 4 colheres de pó de café no coador.
11. Despejar a água lentamente sobre o pó.
12. Aguardar a água escorrer completamente.
13. Retirar o coador.
14. Tampar a garrafa.

Repare no passo 7. Ele não descreve uma ação, e sim uma espera condicionada. Guarde essa sensação: na lição 13 você vai reencontrá-la com o nome de laço de repetição.

Repare também que a ordem é rígida. Executar o passo 11 antes do passo 10 resulta em água quente, não em café.

Exercícios resolvidos

1) Descreva a sequência de ações para escovar os dentes.

1. Ir até a pia.
2. Pegar a escova de dentes.
3. Pegar o tubo de pasta.
4. Abrir a tampa do tubo.
5. Apertar o tubo depositando a pasta sobre as cerdas.
6. Fechar a tampa do tubo.
7. Apoiar o tubo na pia.
8. Abrir a torneira.
9. Molhar as cerdas.
10. Fechar a torneira.
11. Levar a escova até a boca.
12. Escovar os dentes superiores por 30 segundos.
13. Escovar os dentes inferiores por 30 segundos.
14. Retirar a escova da boca.
15. Abrir a torneira.
16. Encher a boca de água.
17. Bochechar.
18. Cuspir a água.
19. Enxaguar a escova.
20. Fechar a torneira.
21. Guardar a escova.

Se a sua lista tinha seis passos, releia a minha e observe onde estão as ações que o seu cérebro pulou. É exatamente esse tipo de omissão que se transforma em erro quando começarmos a escrever código.

2) Descreva a sequência de ações para trocar uma lâmpada queimada.

1. Desligar o interruptor do cômodo.
2. Pegar a lâmpada nova.
3. Posicionar a escada embaixo do bocal.
4. Subir a escada.
5. Girar a lâmpada queimada no sentido anti-horário até soltá-la.
6. Descer a escada.
7. Guardar a lâmpada queimada em local seguro.
8. Subir a escada com a lâmpada nova.
9. Encaixar a lâmpada nova no bocal.
10. Girar a lâmpada no sentido horário até firmar.
11. Descer a escada.
12. Ligar o interruptor.

13. Verificar se a lâmpada acendeu.
14. Guardar a escada.

Note os passos 1 e 13. O primeiro é uma precaução; o último é uma verificação. Programas bem escritos são cheios dos dois.

3) Descreva a sequência de ações para enviar uma mensagem de texto pelo celular.

1. Desbloquear a tela do celular.
2. Localizar o ícone do aplicativo de mensagens.
3. Tocar no ícone do aplicativo.
4. Aguardar o aplicativo abrir.
5. Localizar o contato desejado na lista.
6. Tocar sobre o nome do contato.
7. Tocar no campo de digitação.
8. Digitar o texto da mensagem.
9. Revisar o texto digitado.
10. Tocar no botão de enviar.
11. Confirmar que a mensagem aparece na conversa.

4) Descreva a sequência de ações para fritar um ovo.

1. Pegar a frigideira.
2. Colocar a frigideira sobre a boca do fogão.
3. Acender o fogo em temperatura média.
4. Colocar uma colher de óleo na frigideira.
5. Aguardar o óleo aquecer.
6. Pegar o ovo.
7. Quebrar a casca do ovo na borda da frigideira.
8. Despejar o conteúdo do ovo na frigideira.
9. Descartar a casca.
10. Adicionar sal a gosto.
11. Aguardar a clara ficar branca por completo.
12. Desligar o fogo.
13. Transferir o ovo para o prato com uma espátula.

Exercícios

Escreva as sequências em português, numerando cada passo. Não use código nenhum.

1. Descreva a sequência para sacar dinheiro em um caixa eletrônico.
2. Descreva a sequência para lavar uma louça suja na pia.
3. Descreva a sequência para trocar o pneu furado de um carro.
4. Descreva a sequência para fazer um cadastro em um site.
5. Descreva a sequência para preparar um sanduíche de queijo quente.

6. Descreva a sequência para estacionar um carro em uma vaga paralela.
7. Descreva a sequência para pagar uma conta usando o aplicativo do banco.
8. Descreva a sequência para regar as plantas de uma casa.
9. Pegue o exercício 1 e reescreva quebrando cada passo em passos ainda menores. Você deve conseguir pelo menos o dobro de linhas.
10. Peça para alguém executar literalmente a sequência que você escreveu no exercício 5, sem interpretar nada além do que está escrito. Anote tudo que deu errado.

O exercício 10 é o mais importante da lição. Uma pessoa executando o seu texto ao pé da letra é a melhor simulação de computador que existe.

Dica do dia

O computador não faz o que você quer. Ele faz o que você escreve. Sempre que um programa se comportar de forma estranha, releia o que você escreveu procurando o passo que faltou, e não o passo que está errado.

DIA 02 — PROGRAMAÇÃO DESCRITIVA

Objetivo do dia

Desenvolver a habilidade de reconhecer os 4 tipos de intenções que um programa de computador pode executar.

Introdução

Na aula anterior você aprendeu a detalhar os passos de uma tarefa.

Na aula de hoje você vai aprender que todas as ações que um programa pode executar estão definidas dentro de quatro intenções básicas. Reconhecer essas intenções é o que permite olhar para um problema em português e enxergar, por baixo, o programa que vai resolvê-lo.

Essas quatro intenções valem para qualquer linguagem. Java, JavaScript, Python ou C mudam apenas o nome dos comandos; a intenção por trás deles é sempre a mesma. É por isso que quem aprende a pensar assim troca de linguagem com facilidade depois.

Intenção de apresentação de mensagens

Todos os comandos utilizados para apresentar uma mensagem ao usuário estão dentro desta intenção.

Diferentes linguagens de programação utilizam nomes diferentes para representar a intenção de apresentação de mensagens. Em Java podemos utilizar `System.out.println`, `System.out.print` ou `System.out.printf`. Os três servem para apresentar uma mensagem, mas são usados em contextos diferentes, como você vai aprender na lição 3.

Para a aula de hoje nós vamos definir o pseudocomando “**escrever mensagem**” e vamos utilizá-lo sempre que desejarmos que algum tipo de informação seja apresentada ao usuário.

Intenção de armazenamento de dados

A maioria dos programas de computador precisa, em algum momento, realizar a tarefa de armazenar algum tipo de informação.

Esse armazenamento pode ser temporário, quando ele guarda a informação na memória do computador, ou pode ser persistente, quando ele grava as informações em um arquivo no disco rígido. Na grande maioria das vezes, o armazenamento é feito de forma temporária.

Em Java, o armazenamento temporário acontece nas variáveis, que você vai estudar na lição 4.

Para a aula de hoje nós vamos utilizar o pseudocomando “**armazenar informação**” sempre que desejarmos guardar algum tipo de informação.

Intenção de leitura de dados

Assim como os programas apresentam mensagens para o usuário, eles também precisam ser capazes de realizar a operação de leitura de informações. Quando o usuário digita uma mensagem no teclado, por exemplo, o programa precisa ser capaz de realizar a leitura dessa informação.

Em Java, essa leitura é feita pela classe `Scanner`, que você vai conhecer na lição 5.

Para a nossa aula de hoje utilizaremos o pseudocomando “**ler informação**” sempre que precisarmos descrever uma ação de leitura de dados.

Intenção de processamento de dados

Essa é a principal intenção realizada por um programa de computador: a intenção de processar informação.

Esse processamento pode ser uma série de atividades diferentes, como o processamento de operações matemáticas (lição 5), de operações de comparação (lição 7), de operações lógicas (lição 7), de operações de desvio de fluxo (lições 8 e 9) ou de operações de repetição de comandos (lições 13 e 14).

Para a nossa aula de hoje utilizaremos o pseudocomando “**processar operação**” sempre que precisarmos detalhar algum tipo de processamento.

Escrevendo com os quatro pseudocomandos

Agora que você já conhece as 4 principais intenções de um programa de computador, vamos ver como utilizá-las para descrever as ações de um programa.

Exemplo: um programa que calcula a média de duas notas.

```
escrever mensagem "Digite a primeira nota"
ler informação
```

```
armazenar informação em nota1
escrever mensagem "Digite a segunda nota"
ler informação
armazenar informação em nota2
processar operação (nota1 + nota2) / 2
armazenar informação em media
escrever mensagem media
```

Nove linhas, quatro intenções, nenhuma linguagem de programação envolvida. E, ainda assim, isso já é um programa: falta apenas a tradução.

De olho nos detalhes

Toda leitura vem acompanhada de um armazenamento. Ler uma informação e não guardá-la em lugar nenhum é o mesmo que não ter lido. Sempre que você escrever “ler informação”, a próxima linha deve ser “armazenar informação”.

Todo processamento produz um resultado que precisa de destino. O resultado de um cálculo ou é armazenado, ou é apresentado imediatamente. Não existe terceira opção.

A ordem das intenções revela a estrutura do programa. Praticamente todo programa iniciante segue o mesmo desenho:

```
LER → PROCESSAR → ESCREVER
```

Quando você travar em um exercício mais adiante, volte a esta descrição. Na maioria absoluta das vezes o problema está aqui, e não na sintaxe da linguagem.

Do pseudocomando ao Java

Só para você ver aonde estamos indo, este é o mesmo programa da média escrito em Java. Não se preocupe em entender cada peça agora — as lições 3, 4 e 5 destrincham tudo:

```
import java.util.Scanner;

public class Media {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite a primeira nota");           //
        escrever mensagem
        double nota1 = Double.parseDouble(scanner.nextLine()); // ler
        + armazenar

        System.out.println("Digite a segunda nota");           //
        escrever mensagem
        double nota2 = Double.parseDouble(scanner.nextLine()); // ler
        + armazenar

        double media = (nota1 + nota2) / 2;                     //
        processar + armazenar
```

```

        System.out.println(media);           //
    escrever mensagem

    scanner.close();
}

```

Compare linha a linha com a descrição anterior. A estrutura sobreviveu inteira à tradução — só mudou o idioma.

Exercícios resolvidos

1) Descreva, usando os quatro pseudocomandos, um programa que calcula a área de um retângulo.

```

escrever mensagem "Digite a base"
ler informação
armazenar informação em base
escrever mensagem "Digite a altura"
ler informação
armazenar informação em altura
processar operação base * altura
armazenar informação em area
escrever mensagem area

```

2) Descreva um programa que calcula o troco de uma compra.

```

escrever mensagem "Digite o valor da compra"
ler informação
armazenar informação em valorCompra
escrever mensagem "Digite o valor pago"
ler informação
armazenar informação em valorPago
processar operação valorPago - valorCompra
armazenar informação em troco
escrever mensagem "O troco é:"
escrever mensagem troco

```

3) Descreva um programa que informa se uma pessoa pode dirigir, a partir da idade.

```

escrever mensagem "Digite a sua idade"
ler informação
armazenar informação em idade
processar operação: comparar se idade é maior ou igual a 18
    se verdadeiro, armazenar "pode dirigir" em resposta
    se falso, armazenar "não pode dirigir" em resposta
escrever mensagem resposta

```

Repare que aqui apareceu um processamento de desvio de fluxo. Ele é o assunto da lição 8.

4) Descreva um programa que apresenta a tabuada de um número.

```
escrever mensagem "Digite um número"
ler informação
armazenar informação em numero
armazenar 1 em multiplicador
processar operação de repetição enquanto multiplicador for menor ou
igual a 10
    processar operação numero * multiplicador
    armazenar informação em resultado
    escrever mensagem resultado
    processar operação multiplicador + 1
    armazenar informação em multiplicador
```

Este é um processamento de repetição, assunto das lições 13 e 14. Note que mesmo sem conhecer nenhum comando de repetição, você já consegue descrever a intenção.

Exercícios

Descreva os programas abaixo utilizando apenas os quatro pseudocomandos.

1. Um programa que converte uma temperatura de Celsius para Fahrenheit.
2. Um programa que calcula o valor da parcela de uma compra sem juros.
3. Um programa que calcula o IMC de uma pessoa a partir do peso e da altura.
4. Um programa que apresenta a soma de três números digitados pelo usuário.
5. Um programa que informa se um número digitado é par ou ímpar.
6. Um programa que calcula quanto uma pessoa vai gastar de combustível em uma viagem, a partir da distância, do consumo do carro e do preço do litro.
7. Um programa que apresenta o maior entre dois números digitados.
8. Um programa que calcula o valor final de uma compra com 10% de desconto.
9. Um programa que lê o nome e o ano de nascimento de uma pessoa e apresenta uma mensagem de boas-vindas com a idade dela.
10. Um programa que lê cinco notas e apresenta a média.
11. Um programa que valida uma senha: lê a senha digitada e informa se ela está correta.
12. Um programa que lê o preço de um produto e a quantidade comprada e apresenta o total a pagar.

Dica do dia

Antes de escrever qualquer linha de código, escreva o programa em português. Programador experiente não pula essa etapa — ele apenas a faz mentalmente e rápido. Enquanto você está começando, faça no papel. Você vai economizar horas de depuração.

DIA 03 — PRIMEIROS PROGRAMAS

Objetivo do dia

Criar seus primeiros programas utilizando Java.

Introdução

Na aula anterior você aprendeu sobre os quatro tipos de intenções que um programa pode executar.

Na aula de hoje você vai aprender a criar e executar um programa escrito em Java, e vai conhecer a intenção de apresentação de mensagens na prática.

Pré-requisitos da aula

Para a aula de hoje você vai precisar ter instalado em seu computador os seguintes programas:

- **JDK 21 ou superior** — <https://adoptium.net>
- **Visual Studio Code** — <https://code.visualstudio.com/download>
- **Extension Pack for Java** — extensão do VS Code, instalada pela aba de extensões

Se preferir, você pode usar o **IntelliJ IDEA Community Edition** no lugar do VS Code. Ele é mais pesado, porém tem o melhor suporte a Java do mercado. Para este livro, qualquer um dos dois serve.

Depois de instalar, abra o terminal e digite:

```
java -version
```

Se aparecer algo como `openjdk version "21.0.x"`, está tudo certo. Se aparecer “comando não encontrado”, a instalação não foi concluída ou o computador precisa ser reiniciado.

Como funciona a criação de um programa de computador

O processo para a criação de um programa funciona da seguinte forma:

Primeiro nós escrevemos, em um arquivo, o código do programa. Esse arquivo é chamado de **código-fonte**, ou simplesmente “fonte”. O código-fonte é escrito em uma linguagem de programação, que é uma linguagem formal com um pequeno conjunto de padrões, regras de escrita e comandos. O computador não sabe interpretar o código-fonte diretamente.

O próximo passo é fazer com que o que foi escrito no código-fonte possa ser executado. Isso pode acontecer de duas formas:

1) Interpretação — um programa auxiliar chamado interpretador lê o código-fonte e transmite os comandos ao sistema operacional, linha por linha, no momento da execução.

2) Compilação — um programa auxiliar chamado compilador lê o código-fonte e cria um novo arquivo, chamado de código compilado ou executável, que o sistema operacional sabe como ler e executar.

E o Java, onde entra?

Java faz as duas coisas, e entender isso vai te poupar confusão mais adiante.

```
código-fonte (.java)
    ↓ javac (compilador)
    bytecode (.class)
        ↓ java (JVM — máquina virtual)
    sistema operacional
```

O compilador `javac` lê o seu arquivo `.java` e gera um arquivo `.class` contendo **bytecode** — um código intermediário que não é específico de nenhum sistema operacional. Depois, a **JVM** (Java Virtual Machine) lê esse bytecode e o executa na máquina em que estiver rodando.

É daí que vem o lema histórico da linguagem: *escreva uma vez, execute em qualquer lugar*. O mesmo arquivo `.class` roda em Windows, Linux, Mac ou em um servidor na nuvem, porque quem se adapta ao sistema operacional é a JVM, e não o seu programa.

Há uma consequência prática disso que você vai sentir já hoje: **em Java, uma parte grande dos erros aparece antes de o programa rodar**. O compilador confere o seu código e se recusa a gerar o `.class` enquanto houver problema. Isso incomoda no começo e vira uma rede de proteção depois — vale a pena encarar cada erro de compilação como um aviso gratuito, e não como um obstáculo.

Criando o seu primeiro programa

Passo 01 — Criar a pasta do projeto

Crie uma pasta chamada `dia03` em um local de fácil acesso e abra essa pasta no VS Code (menu *Arquivo* → *Abrir Pasta*).

Diferente de outras linguagens, você **não precisa de nenhum arquivo de configuração** para começar. Não há `package.json`, não há dependências para instalar. Uma pasta e um arquivo bastam.

Passo 02 — Criar o arquivo do programa

Dentro da pasta, crie um arquivo chamado `PrimeiroPrograma.java` e digite o código abaixo:

```
public class PrimeiroPrograma {
    public static void main(String[] args) {
        System.out.println("Olá Mundo");
    }
}
```

Passo 03 — Executar o programa

Abra o terminal integrado do VS Code (menu *Terminal* → *Novo Terminal*) e digite:

```
java PrimeiroPrograma.java
```

O resultado deve ser:

```
Olá Mundo
```

Se preferir fazer as duas etapas separadamente, para ver o bytecode sendo gerado:

```
javac PrimeiroPrograma.java  
java PrimeiroPrograma
```

Repare que o segundo comando **não** leva a extensão `.java`. Ele executa a classe, não o arquivo.

Vamos analisar o código

Linha 1:

```
public class PrimeiroPrograma {
```

Declara uma classe chamada `PrimeiroPrograma`. Em Java, toda linha de código executável precisa morar dentro de uma classe — não existe código solto no arquivo.

O nome da classe segue duas regras obrigatórias: começa com letra maiúscula (convenção) e **precisa ser idêntico ao nome do arquivo** (obrigatório). A classe `PrimeiroPrograma` vive no arquivo `PrimeiroPrograma.java`.

Linha 2:

```
    public static void main(String[] args) {
```

Declara o método `main`, que é o **ponto de entrada** do programa. Quando você manda executar, a JVM procura exatamente este método e começa por ele. Todo programa Java tem um.

Vamos abrir a assinatura, palavra por palavra:

- `public` — indica que o método pode ser acessado de fora da classe. A JVM precisa disso para conseguir chamá-lo.
- `static` — indica que o método pertence à classe, e não a um objeto criado a partir dela. Sem isso, a JVM precisaria criar um objeto antes de executar o programa — e não teria como.
- `void` — indica que o método não devolve nenhum valor.
- `main` — o nome do método. Escrito exatamente assim, em minúsculas.
- `String[] args` — a lista de argumentos que podem ser passados pela linha de comando.

Você vai digitar essa linha centenas de vezes ao longo do livro. Não tente decorá-la hoje; a repetição faz esse trabalho sozinha.

Linha 3:

```
        System.out.println("Olá Mundo");
```

Apresenta a mensagem no console. Esta é a intenção de apresentação de mensagens que você estudou ontem.

Lendo da esquerda para a direita: `System` é uma classe do próprio Java, `out` é a saída padrão (o console) e `println` é o método que escreve e pula uma linha.

Linhas 4 e 5:

```
}  
}
```

Fecham o método `main` e a classe. Em Java, todo bloco aberto com `{` precisa ser fechado com `}`. Manter a indentação alinhada é o que permite enxergar isso rapidamente.

De olho nos detalhes

`print`, `println` e `printf`

```
System.out.print("Olá ");           // escreve e NÃO pula linha  
System.out.print("Mundo");  
System.out.println("!");           // escreve e pula linha  
// saída: Olá Mundo!
```

```
System.out.printf("Olá %s!%n", "Maria"); // escreve com formatação  
// saída: Olá Maria!
```

O `printf` vai ficar muito útil a partir da lição 5, quando começarmos a apresentar valores com casas decimais.

O ponto e vírgula é obrigatório

Toda instrução em Java termina com `;`. Não é opcional, não é estilo: é regra da linguagem. Esquecer o ponto e vírgula é o erro de compilação mais comum da primeira semana, e a mensagem que aparece é `';' expected`.

Aspas simples e aspas duplas são coisas diferentes

```
System.out.println("Olá Mundo"); // correto  
System.out.println('Olá Mundo'); // ERRO de compilação
```

Em Java:

- **aspas duplas** delimitam um `String`, ou seja, um texto de qualquer tamanho;
- **aspas simples** delimitam um `char`, ou seja, exatamente **um** caractere.

Então `'A'` é válido e `'Olá Mundo'` não é. E, diferente de outras linguagens, **não existe crase como delimitador de texto** em Java.

Caracteres especiais dentro do texto

```
System.out.println("Ele disse: \"bom dia\""); // aspas dentro do  
texto  
System.out.println("Primeira linha\nSegunda linha"); // quebra de  
linha  
System.out.println("Nome:\tMaria");           // tabulação  
System.out.println("C:\\Users\\Maria");       // barra invertida
```

A contrabarra \ é o caractere de escape. Ela avisa que o próximo caractere tem significado especial.

Comentários

```
// comentário de uma linha

/*
    comentário
    de várias linhas
*/
```

O compilador ignora comentários por completo. Use-os para explicar **por que** o código faz algo, não **o que** ele faz — o que ele faz já está escrito ali.

Exercícios resolvidos

1) Crie um programa que apresente no console a mensagem “Olá Mundo”.

```
public class Exercicio01 {
    public static void main(String[] args) {
        System.out.println("Olá Mundo");
    }
}
```

2) Crie um programa que apresente o seu nome em uma linha e a sua cidade na linha seguinte.

```
public class Exercicio02 {
    public static void main(String[] args) {
        System.out.println("Maria Oliveira");
        System.out.println("Blumenau");
    }
}
```

3) Crie um programa que apresente a frase “Java é divertido” utilizando três comandos print separados.

```
public class Exercicio03 {
    public static void main(String[] args) {
        System.out.print("Java ");
        System.out.print("é ");
        System.out.println("divertido");
    }
}
```

O último comando é um println para garantir a quebra de linha ao final. Se todos fossem print, o cursor do terminal ficaria colado no fim da frase.

4) Crie um programa que desenhe um quadrado de asteriscos com 5 linhas e 5 colunas.

```
public class Exercicio04 {
    public static void main(String[] args) {
```

```

        System.out.println("*****");
        System.out.println("*****");
        System.out.println("*****");
        System.out.println("*****");
        System.out.println("*****");
    }
}

```

Cinco comandos repetidos incomodam? Ótimo. Guarde esse incômodo até a lição 14, quando você vai resolver o mesmo exercício em três linhas.

5) Crie um programa que apresente a frase Ele disse: "bom dia" no console.

```

public class Exercicio05 {
    public static void main(String[] args) {
        System.out.println("Ele disse: \"bom dia\"");
    }
}

```

Exercícios

1. Crie um programa que apresente a mensagem “Bem-vindo ao mundo Java!”.
2. Crie um programa que apresente, em três linhas, o seu nome, a sua idade e a sua profissão.
3. Crie um programa que apresente as três primeiras linhas da letra de uma música que você goste, usando um `println` para cada linha.
4. Crie um programa que desenhe um retângulo de asteriscos com 3 linhas e 8 colunas.
5. Crie um programa que desenhe um triângulo de asteriscos com 4 linhas, começando com 1 asterisco e terminando com 4.
6. Crie um programa que apresente o seu nome completo usando apenas comandos `print`, com um comando para cada palavra.
7. Crie um programa que apresente uma linha de 30 traços, depois um título centralizado com espaços, depois outra linha de 30 traços.
8. Crie um programa que apresente o caminho `C:\Program Files\Java` no console.
9. Crie um programa que apresente duas linhas de texto usando um único `println` (dica: use `\n`).
10. Crie um programa que apresente uma tabela com três nomes alinhados em colunas, usando `\t`.
11. Renomeie o arquivo do exercício 1 para `Teste.java` sem mudar o nome da classe. Execute, leia a mensagem de erro e depois corrija.
12. Apague um ponto e vírgula do exercício 2 de propósito. Execute, leia a mensagem de erro e depois corrija.

Os exercícios 11 e 12 parecem bobos, mas não são. Provocar o erro de propósito, com calma, ensina a ler mensagens de compilação — habilidade que você vai usar todos os dias da sua vida como programador.

Dica do dia

O nome do arquivo precisa ser igual ao nome da classe pública. Se a mensagem de erro disser `class X is public, should be declared in a file named X.java`, é exatamente isso que aconteceu. Renomeie o arquivo ou a classe, e siga em frente.

DIA 04 — COMO GUARDAR INFORMAÇÕES NA MEMÓRIA DO COMPUTADOR

Objetivo do dia

Criar programas que armazenam informações na memória, escolhendo o tipo de dado correto para cada situação.

Introdução

Na aula anterior você aprendeu a criar e executar um programa em Java e a apresentar mensagens no console.

Na aula de hoje você vai aprender a intenção de armazenamento de dados: como reservar um espaço na memória do computador, dar um nome a esse espaço e guardar uma informação nele.

O que é uma variável

Uma variável é um espaço na memória do computador, identificado por um nome, onde o programa guarda um valor para usar depois.

Em Java, criar uma variável exige três informações:

```
int idade = 30;
//      |      |      |
//      |      |      └─ valor inicial
//      |      └─ identificador (o nome)
//      └─ tipo de dado
```

A leitura da linha é: “reserve um espaço na memória chamado `idade`, que só aceita números inteiros, e guarde o valor 30 nele”.

Essa exigência do tipo é a característica que mais diferencia Java das linguagens que você talvez já tenha visto por aí. Em muitas linguagens, uma variável pode guardar um texto agora e um número daqui a três linhas. **Em Java, o tipo é declarado uma vez e não muda nunca mais.**

Parece burocracia, e no começo é mesmo um pouco. Mas veja o que você ganha em troca: o editor sabe exatamente o que cada variável guarda e consegue sugerir os comandos disponíveis, e o compilador acusa antes da execução qualquer operação incoerente. Uma categoria inteira de erros deixa de existir.

Os tipos de dados

Tipos numéricos inteiros

Representam números sem casas decimais.

```
byte quantidade = 100;           // de -128 a 127
short ano = 2026;                // de -32.768 a 32.767
int populacao = 2000000;         // de -2,1 bilhões a 2,1 bilhões
long distancia = 9800000000L;    // valores gigantescos (note o L no final)
```

Na prática, **use int para praticamente tudo** e recorra ao long apenas quando o valor puder ultrapassar 2 bilhões.

Tipos numéricos decimais

Representam números com casas decimais.

```
float altura = 1.75f;           // menor precisão (note o f no final)
double preco = 29.90;           // maior precisão – use este
```

Na prática, **use sempre double**. O float só compensa em situações muito específicas de economia de memória.

Um detalhe importante do sistema numérico: na programação usamos o padrão inglês, por isso os valores decimais são separados do valor inteiro por um **ponto**, e não por vírgula como no sistema brasileiro. Escreva 29.90, nunca 29,90.

Tipo lógico

O tipo boolean é o nome dado aos valores lógicos, e só pode representar true (verdadeiro) ou false (falso).

```
boolean podeComprar = true;
boolean errou = false;
```

Tipo caractere

O tipo char guarda exatamente um caractere, sempre entre aspas simples.

```
char inicial = 'M';
char nota = 'A';
```

Texto

O tipo String guarda um texto de qualquer tamanho, sempre entre aspas duplas.

```
String nome = "João da Silva";
String mensagem = "Viva a vida!";
```

Repare que String começa com letra maiúscula, enquanto todos os tipos anteriores começam com minúscula. Isso não é casual: os oito tipos anteriores são chamados de **tipos primitivos**, e

String é uma **classe**. É por isso que String tem métodos próprios — assunto da lição 11 — enquanto um int não tem.

Resumo dos tipos que você vai usar

Tipo	Guarda	Exemplo
int	número inteiro	int idade = 30;
long	número inteiro muito grande	long visitas = 9800000000L;
double	número com casas decimais	double preco = 29.90;
boolean	verdadeiro ou falso	boolean ativo = true;
char	um caractere	char inicial = 'M';
String	texto	String nome = "Maria";

Declarar e inicializar

As duas etapas podem acontecer juntas ou separadas:

```
String nome;           // declara: reserva o espaço
nome = "João da Silva"; // inicializa: guarda o primeiro valor

int idade = 30;         // declara e inicializa em uma linha só
```

Depois de inicializada, a variável pode receber novos valores do mesmo tipo:

```
int idade = 30;
idade = 31;           // permitido
idade = "trinta e um"; // ERRO de compilação: tipos incompatíveis
```

O valor null

O valor null indica que uma variável não aponta para nenhum objeto. Ele existe apenas para tipos que são objetos, como String:

```
String nome = null; // permitido
int idade = null;   // ERRO de compilação
```

Um tipo primitivo sempre guarda um valor; ele nunca pode estar “vazio”.

E aqui vale um aviso: em Java **não existe o valor undefined**. Uma variável local declarada e nunca inicializada simplesmente não pode ser usada:

```
int numero;
System.out.println(numero); // ERRO: variable numero might not have
been initialized
```

Mais uma vez, o compilador está te protegendo de um erro que, em outras linguagens, só apareceria durante a execução.

As palavras `var` e `final`

Desde o Java 10, você pode deixar o compilador descobrir o tipo sozinho:

```
var nome = "João";    // o compilador entende que é String
var idade = 30;        // entende que é int
var preco = 29.90;     // entende que é double
```

Atenção a um ponto conceitual: `var` **não** torna a variável flexível. O tipo continua fixo, definido no momento da declaração — você apenas não precisou escrevê-lo. Depois de `var idade = 30`, a linha `idade = "trinta"` continua sendo erro de compilação.

Minha recomendação para as primeiras semanas: **escreva os tipos por extenso**. Ver `String`, `int` e `double` repetidamente ajuda a fixar o modelo mental de tipagem. Depois que isso estiver natural, use `var` à vontade.

Já a palavra `final` cria uma variável que não pode ter o valor alterado, ou seja, uma constante:

```
final double TAXA_ENTREGA = 9.90;
TAXA_ENTREGA = 12.00;    // ERRO de compilação
```

Regras para nomes de variáveis

- Uma variável deve começar com uma letra, sublinhado (`_`) ou cifrão (`$`). Na prática, comece sempre com letra.
- Os caracteres seguintes podem ser letras de “a” até “z”, de “A” até “Z”, dígitos de 0 a 9, `_` ou `$`.
- Não pode ser uma palavra reservada da linguagem (`int`, `class`, `public`, `static`, `new`, entre outras).
- **Java é case sensitive**: `idade`, `Idade` e `IDADE` são três variáveis diferentes.

Além das regras, existem as convenções que toda a comunidade Java segue:

Elemento	Convenção	Exemplo
Variável e método	camelCase	<code>anoNascimento</code> , <code>calcularMedia</code>
Classe	PascalCase	<code>PrimeiroPrograma</code> , <code>ServiceClientes</code>
Constante	MAIÚSCULAS_COM_SUBLINHADO	<code>TAXA_ENTREGA</code>

Seguir a convenção não muda o funcionamento do programa, mas muda a impressão que outro programador tem do seu código. Comece certo.

Vamos analisar o código

Crie o arquivo `Variaveis.java` com o código abaixo:

```

public class Variaveis {
    public static void main(String[] args) {
        // 1
        String nome = "João da Silva";
        // 2
        int idade = 30;
        // 3
        double altura = 1.82;
        // 4
        boolean estudante = true;
        // 5
        char inicial = 'J';
        // 6
        final double TAXA_ENTREGA = 9.90;

        // 7
        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade);
        System.out.println("Altura: " + altura);
        System.out.println("É estudante: " + estudante);
        System.out.println("Inicial: " + inicial);
        System.out.println("Taxa de entrega: " + TAXA_ENTREGA);
    }
}

```

Linha 1: cria uma variável do tipo `String` com o identificador `nome` e armazena o texto "João da Silva".

Linha 2: cria uma variável do tipo `int` com o identificador `idade` e armazena o número inteiro 30.

Linha 3: cria uma variável do tipo `double` com o identificador `altura` e armazena o número decimal 1.82. Repare no ponto separando as casas decimais.

Linha 4: cria uma variável do tipo `boolean` com o identificador `estudante` e armazena o valor lógico `true`.

Linha 5: cria uma variável do tipo `char` com o identificador `inicial` e armazena o caractere 'J', entre aspas simples.

Linha 6: cria uma constante do tipo `double` com o identificador `TAXA_ENTREGA`. Por causa do `final`, esse valor não poderá ser alterado no restante do programa.

Linha 7 e seguintes: apresentam os valores no console. O sinal `+` entre um texto e uma variável faz a **concatenação**: o valor da variável é convertido em texto e emendado na mensagem.

De olho nos detalhes

Concatenação com o sinal +

```
String nome = "Maria";  
int idade = 25;
```

```
System.out.println("Olá " + nome + ", você tem " + idade + " anos.");  
// Olá Maria, você tem 25 anos.
```

Quando um dos lados do + é texto, o Java converte o outro lado em texto e junta os dois. Quando os dois lados são números, ele soma. Isso gera um efeito curioso que vale conhecer desde já:

```
System.out.println("Total: " + 10 + 5);    // Total: 105  
System.out.println("Total: " + (10 + 5));  // Total: 15
```

Na primeira linha, o Java concatenou "Total: " com 10, gerando "Total: 10", e depois concatenou com 5. Na segunda, os parênteses forçaram a soma antes da concatenação. Quando misturar texto e conta na mesma linha, use parênteses.

Escolher o tipo é escolher o limite

```
int quantidade = 3000000000;    // ERRO: valor fora do intervalo do int  
long quantidade = 3000000000L; // correto
```

Escolher um tipo é escolher uma faixa de valores. Quando o número passar de 2 bilhões, use long e acrescente o L no final do literal.

O que o editor mostra é parte do aprendizado

Digite nome . no VS Code e observe a lista de sugestões que aparece. Todos aqueles métodos existem porque nome é um String. Agora digite idade . e veja que nada aparece: int é um tipo primitivo e não tem métodos.

Esse comportamento do editor não é enfeite — é a tipagem estática trabalhando a seu favor. Aproveite-o.

Exercícios resolvidos

1) Crie um programa que use uma variável chamada mensagem, inicializada com o valor “Olá Mundo”, e apresente o valor no console.

```
public class Exercicio01 {  
    public static void main(String[] args) {  
        String mensagem = "Olá Mundo";  
        System.out.println(mensagem);  
    }  
}
```

2) Crie um programa que use uma variável chamada idade, inicializada com o valor 20, e apresente o valor no console.

```
public class Exercicio02 {
    public static void main(String[] args) {
        int idade = 20;
        System.out.println(idade);
    }
}
```

3) Crie um programa que armazene em variáveis o seu nome e a sua idade e apresente as duas informações em uma única mensagem.

```
public class Exercicio03 {
    public static void main(String[] args) {
        String nome = "Maria Oliveira";
        int idade = 28;

        System.out.println("Meu nome é " + nome + " e eu tenho " +
idade + " anos.");
    }
}
```

4) Crie um programa que armazene o preço de um produto e a quantidade comprada, e apresente as duas informações.

```
public class Exercicio04 {
    public static void main(String[] args) {
        String produto = "Teclado mecânico";
        double preco = 249.90;
        int quantidade = 2;

        System.out.println("Produto: " + produto);
        System.out.println("Preço unitário: " + preco);
        System.out.println("Quantidade: " + quantidade);
    }
}
```

5) Crie um programa com uma constante chamada TAXA_JUROS valendo 0.02 e apresente o valor.

```
public class Exercicio05 {
    public static void main(String[] args) {
        final double TAXA_JUROS = 0.02;
        System.out.println("Taxa de juros: " + TAXA_JUROS);
    }
}
```

Exercícios

1. Crie um programa com uma variável chamada mensagem, inicializada com “Bem-vindo ao mundo!”, e apresente o valor no console.
2. Crie um programa com uma variável chamada idade, inicializada com 15.5. Qual tipo você precisou usar? Escreva a resposta em um comentário no próprio arquivo.

3. Crie um programa com uma variável chamada `curso`, inicializada com “JAVA”, e apresente o valor.
4. Crie um programa que armazene em variáveis o seu primeiro nome e o seu sobrenome, e apresente o sobrenome antes do primeiro nome.
5. Crie um programa com uma variável `anoNascimento`, inicializada com o seu ano de nascimento, e apresente o valor.
6. Crie um programa que armazene a inicial do seu nome em um `char` e apresente o valor.
7. Crie um programa que armazene em variáveis o nome de um produto, o preço e se ele está em promoção (`boolean`), e apresente tudo em três linhas.
8. Crie um programa que armazene o seu nome, a sua cidade e o seu estado, e apresente uma frase completa usando concatenação.
9. Crie um programa com uma constante `SALARIO_MINIMO` e uma variável `salario`, e apresente as duas.
10. Crie um programa que declare uma variável `int`, tente atribuir a ela um texto, execute, leia a mensagem de erro e depois corrija.
11. Crie um programa que declare uma variável sem inicializar e tente apresentá-la. Leia a mensagem de erro e depois corrija.
12. Reescreva o exercício 8 usando `var` no lugar dos tipos. O programa continua funcionando?

Dica do dia

Uma variável deve começar com uma letra, sublinhado (`_`) ou cifrão (`$`); os caracteres seguintes podem ser letras, dígitos, `_` ou `$`. Java é case sensitive, ou seja, letras minúsculas são diferentes de maiúsculas. E lembre-se: o nome da variável é a documentação mais lida do seu código. `x` não diz nada; `valorTotalComDesconto` diz tudo.

DIA 05 — TRABALHANDO COM OPERAÇÕES MATEMÁTICAS

Objetivo do dia

Criar programas que leem informações digitadas pelo usuário e realizam operações matemáticas.

Introdução

Na aula anterior você aprendeu a armazenar informações na memória.

Na aula de hoje você vai aprender as duas intenções que faltam: a leitura de dados e o processamento de operações matemáticas. Ao final desta lição você já será capaz de escrever programas completos e úteis.

Os operadores aritméticos

Operador	Operação	Exemplo	Resultado
+	soma	10 + 3	13
-	subtração	10 - 3	7
*	multiplicação	10 * 3	30
/	divisão	10 / 3	3
%	resto da divisão	10 % 3	1

A precedência é a mesma da matemática: primeiro *, / e %, depois + e -. Parênteses alteram a ordem e sempre vencem.

```
System.out.println(2 + 3 * 4);    // 14
System.out.println((2 + 3) * 4);  // 20
```

O operador % (chamado de módulo) devolve o resto de uma divisão inteira. Ele é muito mais útil do que parece:

```
System.out.println(10 % 2);    // 0 → 10 é par
System.out.println(7 % 2);     // 1 → 7 é ímpar
System.out.println(365 % 7);   // 1 → sobra 1 dia após 52 semanas
```

Guarde o teste `numero % 2 == 0` para descobrir se um número é par. Você vai usá-lo muitas vezes.

A divisão inteira: a maior armadilha da lição

Olhe com atenção para a linha da tabela que mostra 10 / 3 resultando em 3.

```
int a = 10;
int b = 3;
System.out.println(a / b);    // 3
```

Em Java, **a divisão entre dois números inteiros é uma divisão inteira**: a parte decimal é simplesmente descartada, sem arredondamento e sem nenhum aviso.

Isso não é um defeito, é uma consequência da tipagem: se os dois operandos são `int`, o resultado também precisa ser `int`.

Existem três formas de obter o resultado decimal, e basta que **um** dos operandos seja `double`:

```
double a = 10;
int b = 3;
System.out.println(a / b);    // 3.3333333333333335

int x = 10, y = 3;
System.out.println((double) x / y); // 3.3333333333333335

System.out.println(10.0 / 3); // 3.3333333333333335
```

Este comportamento é responsável por uma quantidade impressionante de erros em cálculos de média, porcentagem e desconto. Sempre que um resultado decimal aparecer “redondo demais”, desconfie dele e confira os tipos envolvidos.

Conversão de tipos

Converter de um tipo menor para um maior é automático, porque não há perda:

```
int idade = 30;
double idadeDecimal = idade;    // 30.0
```

Converter de um tipo maior para um menor precisa ser pedido explicitamente, porque há perda. Isso se chama **cast**:

```
double preco = 29.90;
int precoInteiro = (int) preco;    // 29 – a parte decimal é cortada
```

Atenção: o cast **trunca**, não arredonda. Para arredondar de verdade, use `Math.round`:

```
System.out.println((int) 29.90);    // 29
System.out.println(Math.round(29.90));    // 30
```

A classe Math

O Java já traz pronta uma classe com as operações matemáticas mais comuns:

```
Math.round(4.6)        // 5        arredonda para o inteiro mais próximo
Math.floor(4.9)        // 4.0      arredonda para baixo
Math.ceil(4.1)         // 5.0      arredonda para cima
Math.abs(-10)          // 10       valor absoluto
Math.pow(2, 10)         // 1024.0   potência
Math.sqrt(81)          // 9.0      raiz quadrada
Math.max(10, 20)        // 20       o maior entre dois valores
Math.min(10, 20)        // 10       o menor entre dois valores
Math.random()           // decimal aleatório entre 0.0 e 1.0
```

Repare que você não precisou criar nada para usar `Math`: os métodos dela são estáticos, exatamente como o `main` que você escreve todos os dias. Na lição 10 isso vai fazer todo o sentido.

Lendo informações digitadas pelo usuário

Até agora todos os valores estavam escritos dentro do código. Para que o programa se torne realmente útil, ele precisa perguntar.

Em Java, a leitura do teclado é feita pela classe `Scanner`, que já vem instalada com o JDK. Não há nada para baixar e nenhuma dependência para configurar.

Crie o arquivo `Soma.java`:

```
import java.util.Scanner;    // 1
```

```

public class Soma {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);           // 2

        System.out.println("Digite o primeiro número:");    // 3
        int numero1 = Integer.parseInt(scanner.nextLine()); // 4

        System.out.println("Digite o segundo número:");
        int numero2 = Integer.parseInt(scanner.nextLine());

        int soma = numero1 + numero2;                       // 5

        System.out.println("A soma é: " + soma);            // 6

        scanner.close();                                     // 7
    }
}

```

Vamos analisar o código

Linha 1:

```
import java.util.Scanner;
```

Carrega a classe Scanner no nosso programa. Ela mora no pacote `java.util`, que faz parte da própria biblioteca do Java.

O `import` sempre aparece **antes** da declaração da classe, no topo do arquivo. Se você esquecer, o compilador acusa `cannot find symbol: class Scanner`. No VS Code, digitar `Scanner` e aceitar a sugestão adiciona o `import` automaticamente.

Linha 2:

```
Scanner scanner = new Scanner(System.in);
```

Cria um objeto Scanner conectado a `System.in`, que representa a entrada padrão do sistema — o teclado.

Repare na palavra `new`: ela é usada sempre que criamos um objeto a partir de uma classe. Você vai reencontrá-la nas lições 12 e 15.

Linha 3:

```
System.out.println("Digite o primeiro número:");
```

Apresenta uma mensagem orientando o usuário. Esse passo é obrigatório na prática: sem ele, o programa fica parado esperando uma digitação sem explicar o que quer.

Linha 4:

```
int numero1 = Integer.parseInt(scanner.nextLine());
```

Esta linha faz três coisas de uma vez, de dentro para fora:

1. `scanner.nextLine()` aguarda o usuário digitar e pressionar Enter, e devolve o que foi digitado como um `String`.
2. `Integer.parseInt(...)` converte esse texto em um número inteiro.
3. O resultado é armazenado na variável `numero1`.

A conversão é necessária porque **tudo que vem do teclado chega como texto**. O texto "10" e o número 10 são coisas diferentes para o Java.

Para os outros tipos, o raciocínio é o mesmo:

```
int i    = Integer.parseInt("30");
double d = Double.parseDouble("29.90");
long l   = Long.parseLong("9800000000");
```

Linha 5: processa a operação e armazena o resultado.

Linha 6: apresenta o resultado, concatenando o número na mensagem.

Linha 7:

```
scanner.close();
```

Libera o recurso utilizado pelo Scanner. Faça isso **uma única vez**, ao final do programa. Fechar o scanner no meio e tentar ler de novo depois provoca erro.

De olho nos detalhes

Por que usamos `nextLine` e não `nextInt`

A classe `Scanner` também tem os métodos `nextInt()` e `nextDouble()`, que leem o valor já convertido. Eles funcionam, porém criam um problema clássico: esses métodos consomem o número, mas **deixam a quebra de linha no buffer**. O próximo `nextLine()` encontra essa quebra pendente, retorna vazio e não espera o usuário digitar nada.

O sintoma é o programa “pulando” uma pergunta. A causa quase nunca é óbvia para quem está começando.

Para evitar essa dor de cabeça por completo, adote uma regra única e mantenha-a pelo livro inteiro: **leia sempre com `nextLine()` e converta com `parseInt` ou `parseDouble`**.

Se o usuário digitar um texto no lugar do número

```
int numero = Integer.parseInt("abc"); // NumberFormatException em execução
```

O programa é encerrado com erro. Tratar isso corretamente é assunto de exceções, que foge do escopo deste livro — mas na lição 15 você vai ver uma forma simples e prática de se proteger.

Formatando números na saída

Cálculos com `double` costumam produzir saídas feias:

```
System.out.println(10.0 / 3); // 3.3333333333333335
```

Para controlar as casas decimais, use printf:

```
System.out.printf("Resultado: %.2f%n", 10.0 / 3); // Resultado: 3,33
```

Os marcadores mais usados são:

Marcador	Significa
%d	número inteiro
%.2f	número decimal com 2 casas
%s	texto
%n	quebra de linha

Exercícios resolvidos

1) Crie um programa que leia dois números e apresente a soma, a subtração, a multiplicação e a divisão.

```
import java.util.Scanner;

public class Exercicio01 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o primeiro número:");
        double numero1 = Double.parseDouble(scanner.nextLine());

        System.out.println("Digite o segundo número:");
        double numero2 = Double.parseDouble(scanner.nextLine());

        System.out.println("Soma: " + (numero1 + numero2));
        System.out.println("Subtração: " + (numero1 - numero2));
        System.out.println("Multiplicação: " + (numero1 * numero2));
        System.out.println("Divisão: " + (numero1 / numero2));

        scanner.close();
    }
}
```

Usei double justamente para que a divisão funcione como esperado. Note também os parênteses em volta de cada conta: sem eles, o Java concatenaria em vez de somar.

2) Crie um programa que leia duas notas e apresente a média.

```
import java.util.Scanner;

public class Exercicio02 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
}
```

```

        System.out.println("Digite a primeira nota:");
        double nota1 = Double.parseDouble(scanner.nextLine());

        System.out.println("Digite a segunda nota:");
        double nota2 = Double.parseDouble(scanner.nextLine());

        double media = (nota1 + nota2) / 2;

        System.out.printf("Média: %.2f%n", media);

        scanner.close();
    }
}

```

Os parênteses em `(nota1 + nota2) / 2` são obrigatórios. Sem eles, o Java dividiria apenas a segunda nota por 2 antes de somar.

3) Crie um programa que leia o valor de uma compra e apresente o valor com 10% de desconto.

```

import java.util.Scanner;

public class Exercicio03 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o valor da compra:");
        double valor = Double.parseDouble(scanner.nextLine());

        double desconto = valor * 0.10;
        double valorFinal = valor - desconto;

        System.out.printf("Desconto: R$ %.2f%n", desconto);
        System.out.printf("Valor final: R$ %.2f%n", valorFinal);

        scanner.close();
    }
}

```

4) Crie um programa que leia uma temperatura em Celsius e apresente o valor em Fahrenheit.

```

import java.util.Scanner;

public class Exercicio04 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite a temperatura em Celsius:");
    }
}

```

```

        double celsius = Double.parseDouble(scanner.nextLine());

        double fahrenheit = celsius * 9.0 / 5 + 32;

        System.out.printf("%.1f°C equivalem a %.1f°F%n", celsius,
fahrenheit);

        scanner.close();
    }
}

```

Repare no 9.0. Se eu tivesse escrito 9 / 5, o Java faria a divisão inteira e o resultado seria 1, transformando toda a fórmula em uma conta errada. Esta é exatamente a armadilha da lição.

5) Crie um programa que leia o ano de nascimento e apresente a idade da pessoa em 2026.

```

import java.util.Scanner;

public class Exercicio05 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o seu ano de nascimento:");
        int anoNascimento = Integer.parseInt(scanner.nextLine());

        int idade = 2026 - anoNascimento;

        System.out.println("Você tem ou fará " + idade + " anos em
2026.");

        scanner.close();
    }
}

```

Exercícios

1. Crie um programa que leia dois números e apresente a soma.
2. Crie um programa que leia a base e a altura de um retângulo e apresente a área.
3. Crie um programa que leia o raio de um círculo e apresente a área (use `Math.PI`).
4. Crie um programa que leia três notas e apresente a média.
5. Crie um programa que leia um valor em reais e a cotação do dólar e apresente o valor convertido.
6. Crie um programa que leia o valor de uma compra e o número de parcelas e apresente o valor de cada parcela.
7. Crie um programa que leia o peso e a altura de uma pessoa e apresente o IMC com duas casas decimais.
8. Crie um programa que leia um número e apresente o resto da divisão dele por 2.

9. Crie um programa que leia a distância de uma viagem, o consumo do carro em km/l e o preço do litro, e apresente o custo total do combustível.
10. Crie um programa que leia o salário de uma pessoa e apresente o valor com 15% de aumento.
11. Crie um programa que leia dois números e apresente a divisão entre eles com quatro casas decimais.
12. Crie um programa que leia um número e apresente o quadrado e a raiz quadrada dele.
13. Crie um programa que leia a quantidade de segundos e apresente quantos minutos e quantos segundos restantes isso representa (dica: use / e %).
14. Crie um programa que leia o preço de um produto e a quantidade, calcule o subtotal, some 9.90 de frete e apresente o total.

Dica do dia

Em Java, 10 / 3 resulta em 3. Sempre que uma conta envolver divisão e você esperar um resultado decimal, garanta que pelo menos um dos números seja `double`. Este é o erro silencioso mais comum entre iniciantes na linguagem.

DIA 06 — CONHEÇA OS OPERADORES COMPOSTOS

Objetivo do dia

Criar programas que atualizam o valor de uma variável a partir do seu valor atual, usando a forma abreviada dos operadores.

Introdução

Na aula anterior você aprendeu a ler dados do teclado e a realizar operações matemáticas.

Na aula de hoje você vai aprender uma forma mais curta de escrever operações que usam o valor atual de uma variável para calcular o novo valor. É uma lição pequena, mas os operadores que você vai aprender hoje aparecem em praticamente todo laço de repetição que você escrever pelo resto da vida.

O problema que eles resolvem

É muito comum precisar atualizar uma variável a partir dela mesma:

```
int total = 100;  
total = total + 50;    // total passa a valer 150
```

Leia a segunda linha com calma, porque ela confunde quem vem da matemática. O sinal `=` não significa “é igual a”: significa “**receba**”. A linha diz “total recebe o valor atual de total mais 50”. Primeiro o lado direito é calculado, depois o resultado é guardado no lado esquerdo.

Como esse padrão se repete o tempo todo, a linguagem oferece uma forma abreviada.

Os operadores compostos

Abreviado	Equivale a	Exemplo
<code>x += 5</code>	<code>x = x + 5</code>	soma 5 ao valor atual
<code>x -= 5</code>	<code>x = x - 5</code>	subtrai 5 do valor atual
<code>x *= 5</code>	<code>x = x * 5</code>	multiplica o valor atual por 5
<code>x /= 5</code>	<code>x = x / 5</code>	divide o valor atual por 5
<code>x %= 5</code>	<code>x = x % 5</code>	guarda o resto da divisão por 5

```
int saldo = 1000;
```

```
saldo += 500;    // 1500
saldo -= 200;    // 1300
saldo *= 2;      // 2600
saldo /= 4;      // 650
```

Os operadores de incremento e decremento

Quando o valor somado ou subtraído é exatamente 1, existe uma forma ainda mais curta:

```
int contador = 0;
```

```
contador++;      // contador passa a valer 1
contador++;      // 2
contador++;      // 3
contador--;      // 2
```

`contador++` é idêntico a `contador += 1`, que por sua vez é idêntico a `contador = contador + 1`.

Esses dois operadores são a base dos laços de repetição das lições 13 e 14. Acostume-se a eles hoje.

Pré-fixado e pós-fixado

Os operadores podem ser escritos antes ou depois da variável, e a diferença aparece quando eles são usados dentro de outra expressão:

```
int a = 5;
System.out.println(a++);    // imprime 5, e DEPOIS incrementa (a fica 6)
```

```
int b = 5;
System.out.println(++b);    // incrementa PRIMEIRO (b fica 6), e imprime 6
```

Na prática: **use sempre a forma pós-fixada (i++) e nunca misture incremento com outras operações na mesma linha.** Código como `a = b++ + ++b;` compila, funciona e é ilegível. Ninguém ganha prêmio por escrever menos caracteres.

Concatenação com +=

O operador += também funciona com texto:

```
String mensagem = "Olá";  
mensagem += " Mundo";  
mensagem += "!";
```

```
System.out.println(mensagem);    // Olá Mundo!
```

Isso será muito útil na lição 12, quando precisarmos montar um texto dentro de uma repetição.

Vamos analisar o código

Crie o arquivo `Carrinho.java`:

```
import java.util.Scanner;  
  
public class Carrinho {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        double total = 0;           // 1  
        int itens = 0;              // 2  
  
        System.out.println("Digite o preço do primeiro produto:");  
        total += Double.parseDouble(scanner.nextLine()); // 3  
        itens++;                     // 4  
  
        System.out.println("Digite o preço do segundo produto:");  
        total += Double.parseDouble(scanner.nextLine());  
        itens++;  
  
        System.out.println("Digite o preço do terceiro produto:");  
        total += Double.parseDouble(scanner.nextLine());  
        itens++;  
  
        System.out.printf("Itens no carrinho: %d\n", itens);  
        System.out.printf("Subtotal: R$ %.2f\n", total);  
  
        total += 9.90;               // 5  
        System.out.printf("Total com frete: R$ %.2f\n", total);  
  
        total *= 0.9;                // 6  
        System.out.printf("Total com 10%% de desconto: R$ %.2f\n",  
total);
```

```
        scanner.close();  
    }  
}
```

Linha 1: cria a variável `total` iniciada em zero. Toda variável que vai acumular valores precisa começar em zero.

Linha 2: cria a variável `itens`, que servirá de contador.

Linha 3: lê o preço digitado e **soma ao total existente**. É o `+=` fazendo o trabalho de `total = total + preco`.

Linha 4: incrementa o contador de itens em 1.

Linha 5: acrescenta o frete ao total.

Linha 6: aplica 10% de desconto multiplicando o total por 0.9.

Repare no `%%` dentro do `printf` da última mensagem. Como o sinal `%` tem significado especial na formatação, para apresentar um sinal de porcentagem literal é preciso escrevê-lo duas vezes.

De olho nos detalhes

Acumulador e contador

O programa acima usa os dois padrões mais comuns da programação, e vale nomeá-los:

- **Acumulador:** variável que soma valores ao longo do programa (`total += preco`).
- **Contador:** variável que conta ocorrências (`itens++`).

Os dois precisam ser inicializados antes do primeiro uso, quase sempre com zero. Esquecer essa inicialização é erro de compilação em Java, felizmente.

Um detalhe surpreendente sobre o cast implícito

```
int total = 10;
```

```
total += 2.5;           // compila! total fica 12  
total = total + 2.5;    // NÃO compila
```

Por que a forma abreviada compila e a longa não? Porque os operadores compostos fazem uma conversão implícita para o tipo da variável da esquerda. O `2.5` foi somado e depois truncado, silenciosamente.

Ou seja: a forma abreviada e a forma longa **não são exatamente equivalentes**. Na prática, evite misturar tipos assim — se a variável precisa guardar decimais, declare-a como `double` desde o início.

Exercícios resolvidos

1) Crie um programa com uma variável `saldo` valendo 1000, debite 250 usando `-=` e apresente o saldo.

```
public class Exercicio01 {
    public static void main(String[] args) {
        double saldo = 1000;
        saldo -= 250;
        System.out.println("Saldo atual: " + saldo);    // 750.0
    }
}
```

2) Crie um programa com uma variável pontos valendo 100 e aplique, em sequência, += 50, -= 30 e *= 2, apresentando o valor após cada operação.

```
public class Exercicio02 {
    public static void main(String[] args) {
        int pontos = 100;

        pontos += 50;
        System.out.println(pontos);    // 150

        pontos -= 30;
        System.out.println(pontos);    // 120

        pontos *= 2;
        System.out.println(pontos);    // 240
    }
}
```

3) Crie um programa que leia o preço de um produto e aumente 15% usando *+=.

```
import java.util.Scanner;

public class Exercicio03 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o preço do produto:");
        double preco = Double.parseDouble(scanner.nextLine());

        preco *= 1.15;

        System.out.printf("Preço com aumento: R$ %.2f%n", preco);

        scanner.close();
    }
}
```

Multiplicar por 1.15 aumenta 15% em uma única operação. Multiplicar por 0.85 reduziria 15%.

4) Crie um programa que leia dois números e troque os valores entre as duas variáveis.

```

import java.util.Scanner;

public class Exercicio04 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o primeiro número:");
        int a = Integer.parseInt(scanner.nextLine());

        System.out.println("Digite o segundo número:");
        int b = Integer.parseInt(scanner.nextLine());

        int auxiliar = a;    // guarda o valor de a
        a = b;               // a recebe b
        b = auxiliar;        // b recebe o valor original de a

        System.out.println("a = " + a);
        System.out.println("b = " + b);

        scanner.close();
    }
}

```

A variável auxiliar é indispensável. Sem ela, a primeira atribuição destruiria o valor de a antes de ele ser copiado.

Exercícios

1. Crie um programa com uma variável contador iniciada em 0, incremente três vezes com ++ e apresente o valor.
2. Crie um programa com uma variável frase valendo "Java" e use += para transformá-la em "Java é divertido".
3. Crie um programa que leia o valor de uma compra, some 9.90 de frete usando += e apresente o total.
4. Crie um programa que leia três preços, acumule todos em uma variável total usando += e apresente o resultado.
5. Crie um programa com uma variável estoque valendo 50, subtraia 12 e depois subtraia mais 8, apresentando o valor a cada passo.
6. Crie um programa que leia um salário e aplique dois aumentos consecutivos de 10%, apresentando o valor final.
7. Crie um programa com `int x = 5` que apresente `x++` e depois `x`, e explique em um comentário o que aconteceu.
8. Crie um programa com `int y = 5` que apresente `++y` e depois `y`, e compare com o exercício anterior.
9. Crie um programa que leia um número e apresente o dobro dele usando `*=`.
10. Crie um programa que leia quatro notas, acumule a soma com `+=`, conte as notas com `++` e apresente a média dividindo um pelo outro.

Dica do dia

i++ é a operação mais escrita da história da programação. Ela parece pequena hoje, mas é ela que faz um laço de repetição avançar. Se um dia o seu programa repetir para sempre, procure primeiro pela linha de incremento que faltou.

DIA 07 — COMO FAZER COMPARAÇÕES

Objetivo do dia

Criar programas que comparam valores e combinam condições, produzindo resultados lógicos.

Introdução

Na aula anterior você aprendeu a atualizar variáveis com os operadores compostos.

Na aula de hoje você vai aprender a comparar valores. Esta é a lição que prepara o terreno para as duas seguintes: sem saber comparar, não há como decidir.

Os operadores de comparação

Toda comparação devolve um valor do tipo `boolean`, ou seja, `true` ou `false`.

Operador	Significado	Exemplo	Resultado
<code>==</code>	igual a	<code>10 == 10</code>	<code>true</code>
<code>!=</code>	diferente de	<code>10 != 5</code>	<code>true</code>
<code>></code>	maior que	<code>10 > 5</code>	<code>true</code>
<code><</code>	menor que	<code>10 < 5</code>	<code>false</code>
<code>>=</code>	maior ou igual a	<code>10 >= 10</code>	<code>true</code>
<code><=</code>	menor ou igual a	<code>4 <= 3</code>	<code>false</code>

```
int idade = 20;
```

```
boolean maiorDeIdade = idade >= 18;    // true
boolean menorDeIdade = idade < 18;      // false
boolean temVinteAnos = idade == 20;     // true
```

Preste atenção na diferença entre `=` e `==`:

- `=` **atribui** um valor a uma variável.
- `==` **compara** dois valores.

Confundir os dois é um erro clássico. A boa notícia é que, em Java, o compilador quase sempre pega essa confusão para você.

A comparação de textos: a lição mais importante do dia

Leia esta seção duas vezes. Ela responde por uma parcela enorme dos erros de quem está aprendendo Java.

```
String a = "java";
String b = new String("java");

System.out.println(a == b);           // false
System.out.println(a.equals(b));      // true
```

Para comparar textos em Java, use sempre o método `.equals()`. Nunca use `==`.

O motivo é conceitual. `String` não é um tipo primitivo, é um objeto. E, quando comparamos objetos:

- `==` pergunta “**são o mesmo objeto na memória?**”
- `.equals()` pergunta “**têm o mesmo conteúdo?**”

Quase sempre o que você quer saber é a segunda coisa.

O que torna esse erro traiçoeiro é que **`o == às vezes funciona`**. A JVM mantém um cache interno de textos literais, então `"java" == "java"` costuma devolver `true`. O programador testa, vê funcionar, confia — e o programa quebra no dia em que o texto vem do `scanner.nextLine()`, que cria um objeto novo a cada leitura.

Duas variações úteis:

```
nome.equals("Maria")           // compara exatamente
nome.equalsIgnoreCase("maria") // ignora maiúsculas e minúsculas
```

E uma regra prática: **use `==` apenas com tipos primitivos** (`int`, `double`, `boolean`, `char`).

Os operadores lógicos

Servem para combinar duas ou mais comparações.

Operador	Nome	Devolve <code>true</code> quando
<code>&&</code>	E	as duas condições são verdadeiras
<code>&& </code>	OU	pelo menos uma condição é verdadeira
<code>!</code>	NÃO	inverte o valor lógico

```
int idade = 25;
boolean temCarteira = true;

boolean podeDirigir = idade >= 18 && temCarteira; // true
boolean estaImpedido = idade < 18 || !temCarteira; // false
```

Tabela-verdade

A	B	A && B	A B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

Uma forma simples de guardar: o && é exigente, basta uma condição falsa para reprovar tudo. O || é generoso, basta uma condição verdadeira para aprovar tudo.

Avaliação curta-circuito

Os operadores && e || param de avaliar assim que o resultado já está definido:

```
if (nome != null && nome.equals("Maria")) {  
    // seguro  
}
```

Se nome for null, a primeira condição é falsa, o Java nem tenta avaliar a segunda e o programa não quebra. Se as condições estivessem invertidas, haveria erro em execução. **A ordem das condições importa.**

Vamos analisar o código

Crie o arquivo Comparacoes.java:

```
import java.util.Scanner;  
  
public class Comparacoes {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.println("Digite o seu nome:");  
        String nome = scanner.nextLine(); // 1  
  
        System.out.println("Digite a sua idade:");  
        int idade = Integer.parseInt(scanner.nextLine());  
  
        System.out.println("Digite o seu saldo:");  
        double saldo = Double.parseDouble(scanner.nextLine());  
  
        boolean maiorDeIdade = idade >= 18; // 2  
        boolean temSaldo = saldo > 0; // 3  
        boolean aprovado = maiorDeIdade && temSaldo; // 4  
        boolean ehAdministrador = nome.equalsIgnoreCase("admin"); // 5  
  
        System.out.println("Maior de idade: " + maiorDeIdade);  
        System.out.println("Tem saldo: " + temSaldo);  
        System.out.println("Aprovado: " + aprovado);
```

```

        System.out.println("É administrador: " + ehAdministrador);
        System.out.println("Não aprovado: " + !aprovado);          // 6

        scanner.close();
    }
}

```

Linha 1: lê o nome digitado. Como é texto, não há conversão.

Linha 2: compara a idade com 18 e armazena o resultado lógico. Repare que guardar o resultado de uma comparação em uma variável `boolean` deixa o código muito mais legível.

Linha 3: verifica se o saldo é positivo.

Linha 4: combina as duas condições com o operador `&&`. Só será `true` se ambas forem verdadeiras.

Linha 5: compara o nome digitado com “admin” usando `equalsIgnoreCase`, que ignora maiúsculas e minúsculas.

Linha 6: apresenta o valor invertido com o operador `!`.

De olho nos detalhes

Não existe valor “verdadeiro por aproximação”

Em algumas linguagens, um texto vazio ou o número zero são considerados “falsos”, e você pode escrever `if (nome)`. **Em Java isso não existe.** Uma condição precisa ser um `boolean` de verdade:

```

String nome = "Maria";

if (nome) { }           // ERRO de compilação
if (!nome.isEmpty()) { } // correto
if (nome != null) { }   // correto

int quantidade = 0;

if (quantidade) { }     // ERRO de compilação
if (quantidade > 0) { }  // correto

```

Isso obriga você a escrever a condição por extenso, e o benefício é que a intenção fica explícita para quem lê.

Java não tem o operador `==`

Se você já viu esse operador em outra linguagem, pode esquecê-lo. Ele existe naquelas linguagens porque elas convertem tipos automaticamente durante comparações. Como Java não faz essas conversões silenciosas, o `==` já se comporta de forma estrita. Um conceito a menos para carregar.

Cuidado com a comparação de números decimais

```
double resultado = 0.1 + 0.2;
System.out.println(resultado == 0.3);    // false!
System.out.println(resultado);           // 0.30000000000000004
```

Isso não é um defeito do Java: é como números decimais são representados em binário, em praticamente toda linguagem. Para valores monetários em sistemas reais, usa-se a classe `BigDecimal`. Para os exercícios deste livro, `double` está de bom tamanho — apenas evite comparar dois decimais com `==`.

Exercícios resolvidos

1) Crie um programa que leia dois números e apresente se eles são iguais e se o primeiro é maior que o segundo.

```
import java.util.Scanner;

public class Exercicio01 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o primeiro número:");
        int a = Integer.parseInt(scanner.nextLine());

        System.out.println("Digite o segundo número:");
        int b = Integer.parseInt(scanner.nextLine());

        System.out.println("São iguais: " + (a == b));
        System.out.println("O primeiro é maior: " + (a > b));

        scanner.close();
    }
}
```

Os parênteses em volta das comparações são necessários. Sem eles, o Java tentaria concatenar antes de comparar.

2) Crie um programa que leia dois textos e compare-os corretamente.

```
import java.util.Scanner;

public class Exercicio02 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o primeiro texto:");
        String texto1 = scanner.nextLine();

        System.out.println("Digite o segundo texto:");
        String texto2 = scanner.nextLine();
    }
}
```

```

        System.out.println("Comparando com equals: " +
texto1.equals(texto2));
        System.out.println("Comparando com ==:      " + (texto1 ==
texto2));

        scanner.close();
    }
}

```

Execute este programa e digite exatamente o mesmo texto nas duas perguntas. O `equals` devolve `true` e o `==` devolve `false`. Guarde essa imagem.

3) Crie um programa que leia a idade e o saldo de uma pessoa e informe se ela tem 18 anos ou mais e saldo acima de 1000.

```

import java.util.Scanner;

public class Exercicio03 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite a idade:");
        int idade = Integer.parseInt(scanner.nextLine());

        System.out.println("Digite o saldo:");
        double saldo = Double.parseDouble(scanner.nextLine());

        boolean aprovado = idade >= 18 && saldo > 1000;

        System.out.println("Aprovado: " + aprovado);

        scanner.close();
    }
}

```

4) Crie um programa que leia um número e informe se ele está entre 10 e 20, incluindo os extremos.

```

import java.util.Scanner;

public class Exercicio04 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite um número:");
        int numero = Integer.parseInt(scanner.nextLine());

        boolean dentroDaFaixa = numero >= 10 && numero <= 20;
    }
}

```

```
        System.out.println("Está entre 10 e 20: " + dentroDaFaixa);  
        scanner.close();  
    }  
}
```

Não existe em Java o formato `10 <= numero <= 20` que usamos na matemática. Toda faixa precisa ser escrita como duas comparações unidas por `&&`.

Exercícios

1. Crie um programa que leia a idade e apresente o resultado de `idade >= 18`.
2. Crie um programa que leia um nome e verifique se ele é igual a “admin” ignorando maiúsculas e minúsculas.
3. Crie um programa que leia um número e apresente se ele é par, usando `%` e `==`.
4. Crie um programa que leia três números e apresente `true` se pelo menos um deles for negativo.
5. Crie um programa que leia dois números e apresente `true` se os dois forem positivos.
6. Crie um programa que leia um usuário e uma senha e apresente `true` somente se forem “admin” e “1234”.
7. Crie um programa que leia uma idade e apresente `true` se a pessoa NÃO for maior de idade, usando o operador `!`.
8. Crie um programa que leia a nota de um aluno e apresente `true` se ela estiver entre 0 e 10.
9. Crie um programa que leia um ano e apresente `true` se ele for bissexto (divisível por 4 e não por 100, ou divisível por 400).
10. Crie um programa que leia dois textos e apresente se eles são diferentes, usando `equals` e o operador `!`.
11. Crie um programa que leia a idade e o peso de uma pessoa e apresente `true` se ela puder doar sangue (entre 18 e 65 anos e no mínimo 50 kg).
12. Crie um programa que leia um e-mail e apresente `true` se ele tiver mais de 5 caracteres e não for igual a “teste”.

Dica do dia

Texto se compara com `.equals()`, nunca com `==`. Escreva isso em um papel e cole no monitor durante a primeira semana. Quando um programa que compara textos “não funciona sem motivo”, esta é a causa em nove de cada dez vezes.

DIA 08 — DESVIO DE FLUXO COM OS COMANDOS IF/ELSE

Objetivo do dia

Criar programas que executam tarefas diferentes dependendo da informação recebida.

Introdução

Na aula anterior você aprendeu a comparar valores e a combinar condições.

Na aula de hoje você vai usar essas comparações para tomar decisões. Até agora, todos os seus programas rodaram sempre da mesma forma, de cima para baixo. A partir de hoje, eles passam a ter caminhos diferentes.

O comando if

O `if` executa um bloco de código **somente se** uma condição for verdadeira.

```
if (condicao) {  
    // executa quando a condição é true  
}
```

Exemplo:

```
int idade = 20;  
  
if (idade >= 18) {  
    System.out.println("Você é maior de idade.");  
}
```

Quando a condição é falsa, o bloco inteiro é ignorado e o programa continua na linha seguinte ao fechamento das chaves.

O comando else

O `else` define o que fazer quando a condição é falsa:

```
if (idade >= 18) {  
    System.out.println("Você é maior de idade.");  
} else {  
    System.out.println("Você é menor de idade.");  
}
```

Só um dos dois blocos executa. Nunca os dois, nunca nenhum.

Encadeando condições com else if

Quando existem mais de dois caminhos possíveis:

```
if (nota >= 9) {  
    System.out.println("Conceito A");  
} else if (nota >= 7) {  
    System.out.println("Conceito B");  
} else if (nota >= 5) {  
    System.out.println("Conceito C");  
} else {  
    System.out.println("Reprovado");  
}
```

A ordem das condições é decisiva. As condições são testadas de cima para baixo e a primeira verdadeira vence — as demais nem chegam a ser avaliadas.

Se você invertesse a ordem começando por `nota >= 5`, um aluno com nota 10 receberia conceito C, porque `10 >= 5` é verdadeiro e o encadeamento pararia ali. Ao trabalhar com faixas de valores, comece sempre pela condição mais restritiva.

O if aninhado

Um `if` pode conter outro dentro dele:

```
if (temConta) {
    if (saldo >= valor) {
        System.out.println("Compra aprovada");
    } else {
        System.out.println("Saldo insuficiente");
    }
} else {
    System.out.println("Cliente sem conta cadastrada");
}
```

Uma orientação de estilo: mais de dois níveis de aninhamento costumam indicar que a lógica pode ser reescrita de forma mais plana, geralmente usando `&&` ou o comando `switch` da próxima lição.

O operador ternário

Para decisões simples, que apenas escolhem entre dois valores:

```
String resultado = idade >= 18 ? "maior de idade" : "menor de idade";
System.out.println(resultado);
```

Leia como: “se `idade >= 18`, use o primeiro valor; senão, use o segundo”.

Use o ternário apenas quando a linha inteira couber confortavelmente na tela. Para qualquer coisa mais complexa, o `if` tradicional é mais legível.

Vamos analisar o código

Crie o arquivo `Aprovacao.java`:

```
import java.util.Scanner;

public class Aprovacao {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o nome do aluno:");
        String aluno = scanner.nextLine(); // 1

        System.out.println("Digite a primeira nota:");
```

```

double nota1 = Double.parseDouble(scanner.nextLine());

System.out.println("Digite a segunda nota:");
double nota2 = Double.parseDouble(scanner.nextLine());

double media = (nota1 + nota2) / 2; // 2

System.out.printf("Média de %s: %.2f%n", aluno, media);

if (media >= 7) { // 3
    System.out.println("Situação: APROVADO");
} else if (media >= 5) { // 4
    System.out.println("Situação: RECUPERAÇÃO");
    double pontosFaltando = 7 - media; // 5
    System.out.printf("Faltaram %.2f pontos.%n",
pontosFaltando);
} else { // 6
    System.out.println("Situação: REPROVADO");
}

scanner.close();
}

```

Linha 1: lê o nome do aluno, que será usado apenas na apresentação.

Linha 2: processa a média das duas notas.

Linha 3: testa a primeira condição. Se a média for 7 ou mais, apresenta a aprovação e **todo o restante do encadeamento é ignorado**.

Linha 4: só é avaliada se a condição anterior for falsa. Neste ponto, o programa já sabe que a média é menor que 7, então basta verificar se é maior ou igual a 5.

Linha 5: repare que esta variável foi declarada **dentro** do bloco `else if`. Ela só existe ali dentro; tentar usá-la fora das chaves é erro de compilação. Isso se chama escopo de bloco.

Linha 6: o `else` final captura todos os casos restantes — aqui, média menor que 5.

De olho nos detalhes

O erro do sinal de igual

```
if (idade = 18) { } // ERRO de compilação em Java
```

Em várias linguagens esse erro é silencioso e devastador: `=` atribui o valor, a expressão resulta em 18, o 18 é considerado “verdadeiro” e o `if` sempre executa. Em Java, como a condição precisa ser um boolean e `idade = 18` produz um `int`, o compilador reprova imediatamente.

Tome cuidado, porém, com variáveis do tipo `boolean`:

```
boolean ativo = false;

if (ativo = true) {    // compila e SEMPRE executa
    System.out.println("Ativo");
}
```

Aqui o compilador não pode ajudar. A solução é simples: nunca compare um boolean com true ou false. Escreva apenas if (ativo) ou if (!ativo).

Chaves são tecnicamente opcionais — use sempre assim mesmo

```
if (idade >= 18)
    System.out.println("Maior de idade");
    System.out.println("Pode dirigir");    // SEMPRE executa
```

Sem chaves, o if controla apenas a instrução imediatamente seguinte. A segunda linha está indentada como se fizesse parte do bloco, mas a indentação engana o leitor, não o compilador.

Use chaves sempre, mesmo para uma única linha. Isso evita o erro acima e evita um erro futuro, quando você precisar acrescentar a segunda linha.

Nem todo if precisa de else

```
if (valor > 500) {
    valor -= valor * 0.10;
}
System.out.println(valor);
```

Quando não há nada a fazer no caso falso, simplesmente não escreva o else.

Exercícios resolvidos

1) Crie um programa que leia um número e informe se ele é positivo, negativo ou zero.

```
import java.util.Scanner;

public class Exercicio01 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite um número:");
        double numero = Double.parseDouble(scanner.nextLine());

        if (numero > 0) {
            System.out.println("Positivo");
        } else if (numero < 0) {
            System.out.println("Negativo");
        } else {
            System.out.println("Zero");
        }

        scanner.close();
    }
}
```

```
}  
}
```

2) Crie um programa que leia dois números e apresente o maior deles.

```
import java.util.Scanner;  
  
public class Exercicio02 {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.println("Digite o primeiro número:");  
        int a = Integer.parseInt(scanner.nextLine());  
  
        System.out.println("Digite o segundo número:");  
        int b = Integer.parseInt(scanner.nextLine());  
  
        if (a > b) {  
            System.out.println("O maior é: " + a);  
        } else if (b > a) {  
            System.out.println("O maior é: " + b);  
        } else {  
            System.out.println("Os números são iguais.");  
        }  
  
        scanner.close();  
    }  
}
```

Repare que tratei o caso de igualdade. Um programa que só testa $a > b$ e assume b no `else` daria uma resposta incorreta quando os números fossem iguais.

3) Crie um programa que leia um usuário e uma senha e apresente “Acesso liberado” se forem “admin” e “1234”.

```
import java.util.Scanner;  
  
public class Exercicio03 {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.println("Usuário:");  
        String usuario = scanner.nextLine();  
  
        System.out.println("Senha:");  
        String senha = scanner.nextLine();  
  
        if (usuario.equals("admin") && senha.equals("1234")) {  
            System.out.println("Acesso liberado");  
        } else {
```

```

        System.out.println("Usuário ou senha inválidos");
    }

    scanner.close();
}

```

Note o uso do `.equals()`. Com `==`, este programa nunca liberaria o acesso, porque os textos vêm do teclado.

4) Crie um programa que leia o valor de uma compra e aplique 10% de desconto acima de 500, ou 5% acima de 200.

```

import java.util.Scanner;

public class Exercicio04 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o valor da compra:");
        double valor = Double.parseDouble(scanner.nextLine());

        double desconto;

        if (valor > 500) {
            desconto = valor * 0.10;
        } else if (valor > 200) {
            desconto = valor * 0.05;
        } else {
            desconto = 0;
        }

        System.out.printf("Desconto: R$ %.2f%n", desconto);
        System.out.printf("Total: R$ %.2f%n", valor - desconto);

        scanner.close();
    }
}

```

Repare que a variável `desconto` foi declarada antes do `if` e recebe valor em todos os caminhos. Se algum caminho não atribuísse valor a ela, o compilador acusaria erro ao usá-la depois.

5) Crie um programa que leia três números e apresente o maior deles.

```

import java.util.Scanner;

public class Exercicio05 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
}

```

```

System.out.println("Digite o primeiro número:");
int a = Integer.parseInt(scanner.nextLine());
System.out.println("Digite o segundo número:");
int b = Integer.parseInt(scanner.nextLine());
System.out.println("Digite o terceiro número:");
int c = Integer.parseInt(scanner.nextLine());

int maior = a;

if (b > maior) {
    maior = b;
}
if (c > maior) {
    maior = c;
}

System.out.println("O maior é: " + maior);

scanner.close();
}

```

Esta é uma técnica que vale memorizar: assuma que o primeiro valor é o maior e vá substituindo conforme encontrar valores maiores. Ela funciona para 3 números e continuará funcionando para 3 mil, quando você aprender listas e repetição.

Exercícios

1. Crie um programa que leia a idade e informe se a pessoa pode votar (16 anos ou mais).
2. Crie um programa que leia uma nota de 0 a 10 e informe “Aprovado” (7 ou mais), “Recuperação” (5 ou mais) ou “Reprovado”.
3. Crie um programa que leia um ano e informe se ele é bissexto.
4. Crie um programa que leia a idade e o peso de uma pessoa e informe se ela pode doar sangue.
5. Crie um programa que leia um número de 1 a 7 e apresente o dia da semana correspondente usando `if/else if`.
6. Crie um programa que leia o salário de uma pessoa e calcule o imposto: isento até 2000, 10% até 5000 e 20% acima disso.
7. Crie um programa que leia três lados e informe se eles formam um triângulo (cada lado deve ser menor que a soma dos outros dois).
8. Crie um programa que leia a temperatura e apresente “Frio” (abaixo de 15), “Ameno” (de 15 a 25) ou “Quente” (acima de 25).
9. Crie um programa que leia dois números e apresente qual é o menor, tratando também o caso de igualdade.
10. Crie um programa que leia o nome de um produto e a quantidade em estoque e apresente um aviso quando o estoque estiver abaixo de 10.

11. Crie um programa que leia a hora do dia (0 a 23) e apresente “Bom dia”, “Boa tarde” ou “Boa noite”.
12. Crie um programa que leia um número e informe se ele é par ou ímpar.
13. Crie um programa que leia o valor da compra e a forma de pagamento (“dinheiro” ou “cartão”) e aplique 5% de desconto apenas no dinheiro.
14. Crie um programa que leia a idade de duas pessoas e informe qual delas é mais velha, tratando o empate.

Dica do dia

A ordem das condições em um encadeamento `if/else if` muda o resultado.

Ao trabalhar com faixas de valores, comece sempre pela condição mais restritiva e vá afrouxando. E use chaves em todos os blocos, mesmo nos de uma linha só.

DIA 09 — DESVIO DE FLUXO COM O COMANDO SWITCH/CASE

Objetivo do dia

Criar programas que escolhem entre muitos caminhos possíveis comparando uma variável com valores fixos.

Introdução

Na aula anterior você aprendeu a desviar o fluxo do programa com `if` e `else`.

Na aula de hoje você vai aprender um segundo comando de desvio, mais adequado para uma situação específica: quando o programa compara **a mesma variável** com **vários valores exatos**.

Quando usar `switch` e quando usar `if`

Este encadeamento funciona, mas é repetitivo:

```
if (dia == 1) {
    System.out.println("Domingo");
} else if (dia == 2) {
    System.out.println("Segunda-feira");
} else if (dia == 3) {
    System.out.println("Terça-feira");
}
```

Repare no padrão: sempre a mesma variável, sempre o operador `==`, sempre um valor fixo. É exatamente para esse caso que existe o `switch`.

A regra é simples:

- **`switch`** quando você compara uma variável com valores exatos.

- **if** quando há faixas de valores, comparações com > e <, ou condições combinadas.

Não existe `case dia > 10`. Faixas continuam sendo trabalho do `if`.

A forma clássica

```
switch (dia) {
    case 1:
        System.out.println("Domingo");
        break;
    case 2:
        System.out.println("Segunda-feira");
        break;
    case 3:
        System.out.println("Terça-feira");
        break;
    default:
        System.out.println("Número inválido");
}
```

O `default` funciona como o `else`: executa quando nenhum `case` corresponde.

O comando `break` encerra o `switch`. E ele é obrigatório — veja por quê.

O break esquecido

Sem o `break`, a execução **continua nos cases seguintes**, ignorando as comparações:

```
switch (dia) {
    case 1:
        System.out.println("Domingo");
        // faltou o break
    case 2:
        System.out.println("Segunda-feira");
        break;
}
// digitando 1, o programa apresenta as DUAS mensagens
```

Esse comportamento se chama *fall-through*. Ele existe de propósito na linguagem, e às vezes é útil para agrupar casos:

```
switch (mes) {
    case 1: case 3: case 5: case 7: case 8: case 10: case 12:
        System.out.println("31 dias");
        break;
    case 4: case 6: case 9: case 11:
        System.out.println("30 dias");
        break;
    case 2:
        System.out.println("28 ou 29 dias");
        break;
}
```

Mas, na prática, nove em cada dez fall-throughs são esquecimento. E como o programa não dá erro nenhum, o problema só aparece nos testes — ou pior, no uso real.

Foi para resolver isso que o Java ganhou uma forma moderna.

A forma moderna: switch com seta

Disponível a partir do Java 14, e a forma que recomendo para tudo que você escrever a partir de hoje:

```
switch (dia) {  
    case 1 -> System.out.println("Domingo");  
    case 2 -> System.out.println("Segunda-feira");  
    case 3 -> System.out.println("Terça-feira");  
    case 4 -> System.out.println("Quarta-feira");  
    case 5 -> System.out.println("Quinta-feira");  
    case 6 -> System.out.println("Sexta-feira");  
    case 7 -> System.out.println("Sábado");  
    default -> System.out.println("Número inválido");  
}
```

Não existe break, não existe fall-through accidental. Para agrupar valores, separe-os por vírgula:

```
case 1, 7 -> System.out.println("Fim de semana");  
case 2, 3, 4, 5, 6 -> System.out.println("Dia útil");
```

Quando um caso precisa de mais de uma instrução, use chaves:

```
case 1 -> {  
    System.out.println("Domingo");  
    System.out.println("Bom descanso!");  
}
```

O switch que devolve um valor

Esta é a forma mais elegante, e provavelmente a que você mais vai usar:

```
String nomeDoDia = switch (dia) {  
    case 1 -> "Domingo";  
    case 2 -> "Segunda-feira";  
    case 3 -> "Terça-feira";  
    case 4 -> "Quarta-feira";  
    case 5 -> "Quinta-feira";  
    case 6 -> "Sexta-feira";  
    case 7 -> "Sábado";  
    default -> "Número inválido";  
};  
  
System.out.println(nomeDoDia);
```

Repare no ponto e vírgula depois da chave de fechamento. Aqui o `switch` não é um comando, e sim uma expressão que produz um valor — por isso a linha termina como qualquer atribuição.

Switch com texto

Funciona desde o Java 7, e a comparação é feita por conteúdo, equivalente ao `.equals()`:

```
System.out.println("Digite a opção:");
String opcao = scanner.nextLine();

switch (opcao) {
    case "cadastrar" -> System.out.println("Abrindo o cadastro...");
    case "listar"     -> System.out.println("Listando os
registros...");
    case "sair"       -> System.out.println("Encerrando o
programa...");
    default           -> System.out.println("Opção inválida");
}
```

O `switch` aceita `int`, `char`, `String` e `enum`. Não aceita `double` nem `boolean`.

Vamos analisar o código

Crie o arquivo `Calculadora.java`:

```
import java.util.Scanner;

public class Calculadora {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o primeiro número:");
        double a = Double.parseDouble(scanner.nextLine()); // 1

        System.out.println("Digite o operador (+, -, *, /):");
        String operador = scanner.nextLine(); // 2

        System.out.println("Digite o segundo número:");
        double b = Double.parseDouble(scanner.nextLine());

        switch (operador) { // 3
            case "+" -> System.out.printf("Resultado: %.2f%n", a + b);
            case "-" -> System.out.printf("Resultado: %.2f%n", a - b);
            case "*" -> System.out.printf("Resultado: %.2f%n", a * b);
            case "/" -> { // 4
                if (b == 0) {
```

```

        System.out.println("Divisão por zero não é
permitida.");
    } else {
        System.out.printf("Resultado: %.2f%n", a / b);
    }
}
default -> System.out.println("Operador inválido.");
// 5
}

scanner.close();
}

```

Linha 1: lê o primeiro número como `double`, para que a divisão funcione corretamente.

Linha 2: lê o operador como texto. Poderia ser `char`, mas `String` é mais simples de ler do teclado.

Linha 3: inicia o `switch` comparando o operador digitado.

Linha 4: este caso precisa de mais de uma instrução, por isso está entre chaves. Dentro dele há um `if` verificando a divisão por zero — repare como os comandos das lições anteriores se combinam naturalmente.

Linha 5: o `default` captura qualquer coisa que o usuário digite fora dos quatro operadores previstos. **Sempre escreva um `default`.** Programa que não trata a entrada inesperada dá a impressão de que travou.

De olho nos detalhes

O `switch` não substitui o `if`

```

switch (idade) {
    case idade >= 18 -> ...    // ERRO: não é permitido
}

```

Comparações com `>`, `<` ou condições combinadas continuam sendo trabalho do `if`. O `switch` só compara igualdade.

Cuidado com texto vindo do teclado

Se o usuário digitar um espaço a mais, o `case` não corresponde. Uma limpeza simples resolve:

```
String opcao = scanner.nextLine().strip().toLowerCase();
```

O método `strip()` remove espaços das pontas e `toLowerCase()` deixa tudo em minúsculas. Você vai estudá-los em detalhe na lição 11.

Prefira sempre a forma com seta

Se você encontrar código antigo com `case 1:` e `break`, saiba lê-lo — há muito código assim no mundo. Mas escreva o seu com `case 1 ->`. É mais curto, mais seguro e não tem como esquecer nada.

Exercícios resolvidos

1) Crie um programa que leia um número de 1 a 7 e apresente o dia da semana.

```
import java.util.Scanner;

public class Exercicio01 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite um número de 1 a 7:");
        int dia = Integer.parseInt(scanner.nextLine());

        switch (dia) {
            case 1 -> System.out.println("Domingo");
            case 2 -> System.out.println("Segunda-feira");
            case 3 -> System.out.println("Terça-feira");
            case 4 -> System.out.println("Quarta-feira");
            case 5 -> System.out.println("Quinta-feira");
            case 6 -> System.out.println("Sexta-feira");
            case 7 -> System.out.println("Sábado");
            default -> System.out.println("Número inválido.");
        }

        scanner.close();
    }
}
```

2) Crie um programa que leia uma letra e informe se ela é vogal ou consoante.

```
import java.util.Scanner;

public class Exercicio02 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite uma letra:");
        String letra = scanner.nextLine().strip().toLowerCase();

        switch (letra) {
            case "a", "e", "i", "o", "u" ->
System.out.println("Vogal");
            default -> System.out.println("Consoante");
        }
    }
}
```

```

        scanner.close();
    }
}

```

Cinco valores agrupados em um único case, separados por vírgula.

3) Crie um programa que leia a sigla de um estado e apresente o nome por extenso, usando o switch como expressão.

```

import java.util.Scanner;

public class Exercicio03 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite a sigla do estado:");
        String sigla = scanner.nextLine().strip().toUpperCase();

        String estado = switch (sigla) {
            case "SC" -> "Santa Catarina";
            case "SP" -> "São Paulo";
            case "RJ" -> "Rio de Janeiro";
            case "MG" -> "Minas Gerais";
            case "RS" -> "Rio Grande do Sul";
            case "PR" -> "Paraná";
            default -> "Estado não cadastrado";
        };

        System.out.println(estado);

        scanner.close();
    }
}

```

4) Crie um programa com um menu de banco que apresente a mensagem correspondente à opção escolhida.

```

import java.util.Scanner;

public class Exercicio04 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("1 - Depositar");
        System.out.println("2 - Sacar");
        System.out.println("3 - Consultar saldo");
        System.out.println("4 - Sair");
        System.out.print("Escolha uma opção: ");
    }
}

```

```

        int opcao = Integer.parseInt(scanner.nextLine());

        switch (opcao) {
            case 1 -> System.out.println("Abrindo a tela de
depósito...");
            case 2 -> System.out.println("Abrindo a tela de
saque...");
            case 3 -> System.out.println("Seu saldo é R$ 1.250,00");
            case 4 -> System.out.println("Encerrando. Até logo!");
            default -> System.out.println("Opção inválida.");
        }

        scanner.close();
    }
}

```

Guarde este exercício: na lição 13 vamos envolvê-lo em um laço de repetição, e ele se transformará em um menu de verdade, que só encerra quando o usuário escolher sair.

Exercícios

1. Crie um programa que leia um número de 1 a 12 e apresente o nome do mês.
2. Crie um programa que leia um mês e apresente quantos dias ele tem.
3. Crie um programa que leia um número de 1 a 5 e apresente a avaliação correspondente, de “Péssimo” até “Excelente”.
4. Crie um programa que leia o nome de uma fruta e apresente o preço por quilo, com pelo menos quatro frutas cadastradas.
5. Crie um programa que leia uma nota de A a E e apresente a faixa de pontuação correspondente.
6. Crie um programa que leia um número de 1 a 7 e informe se o dia é útil ou fim de semana, agrupando os casos.
7. Crie um programa que leia a inicial de um sexo (“M” ou “F”) e apresente uma mensagem personalizada.
8. Crie um programa que leia o código de um produto e apresente o nome dele, usando o `switch` como expressão.
9. Crie um programa que leia a sigla de uma operação (“soma”, “sub”, “mult”, “div”) e dois números, e apresente o resultado.
10. Crie um programa que leia um número de 1 a 4 e apresente a estação do ano correspondente.
11. Reescreva o exercício 5 da lição 8 (dia da semana com `if`) usando `switch` e compare os dois códigos.
12. Crie um programa de menu de uma lanchonete com cinco opções, apresentando o preço do item escolhido.

Dica do dia

Escreva sempre um default. Ele é a rede de segurança do seu programa: cobre tudo aquilo que você não previu, e evita que o usuário fique olhando para uma tela que não respondeu nada.

DIA 10 — TRABALHANDO COM MÉTODOS

Objetivo do dia

Estruturar o código em blocos reutilizáveis, com nome, parâmetros e retorno.

Introdução

Nas lições anteriores todos os nossos programas cresceram dentro do método `main`. Isso funciona enquanto o programa é pequeno, mas rapidamente se transforma em um bloco gigante e difícil de entender.

Na aula de hoje você vai aprender a dividir o programa em pequenas partes com responsabilidades próprias. Em Java, essas partes se chamam **métodos**.

O que é um método

Um método é um bloco de código com nome, que pode receber informações, executar uma tarefa e devolver um resultado.

Você já usa métodos desde a lição 3: `System.out.println` é um método, `Integer.parseInt` é um método, `Math.round` é um método. A partir de hoje você vai criar os seus.

```
static int somar(int a, int b) {  
    return a + b;  
}
```

Vamos abrir a declaração:

```
static int somar (int a, int b) {  
// |      |      |      |  
// |      |      |      |  parâmetros: as informações que o método  
// |      |      |      |  recebe  
// |      |      |      |  nome do método  
// |      |      |      |  tipo do valor devolvido  
// |      |      |      |  modificador
```

E para usá-lo:

```
int resultado = somar(10, 5);  // resultado recebe 15
```

O tipo de retorno

Todo método declara o tipo do valor que devolve:

```
static int somar(int a, int b) {  
    return a + b;           // devolve um int  
}  
  
static double calcularMedia(double n1, double n2) {  
    return (n1 + n2) / 2;    // devolve um double  
}  
  
static String saudacao(String nome) {  
    return "Olá " + nome + "!"; // devolve um String  
}  
  
static boolean ehPar(int numero) {  
    return numero % 2 == 0;    // devolve um boolean  
}
```

Quando o método não devolve nada, o tipo é void:

```
static void apresentarBoasVindas(String nome) {  
    System.out.println("Olá " + nome + ", seja bem-vindo!");  
}
```

Agora a assinatura que você digita desde a lição 3 faz sentido: `public static void main` é um método chamado `main`, que não devolve nada (`void`) e não pertence a nenhum objeto (`static`).

Por que static

Java tem dois tipos de método:

- **Métodos estáticos** (`static`): pertencem à classe. São chamados diretamente, sem criar nada antes. `Math.round(4.6)` é assim.
- **Métodos de instância** (sem `static`): pertencem a um objeto criado a partir da classe. `scanner.nextLine()` é assim — foi preciso criar o `scanner` antes.

Como o `main` é estático, **ele só consegue chamar diretamente outros métodos estáticos**. Por isso, enquanto você não estudar orientação a objetos, marque todos os seus métodos auxiliares como `static`.

Se esquecer, o compilador avisa com a mensagem `non-static method cannot be referenced from a static context`. Agora você sabe exatamente o que ela quer dizer.

Vamos analisar o código

Crie o arquivo `Boletim.java`:

```

import java.util.Scanner;

public class Boletim {

    1    static double calcularMedia(double nota1, double nota2) {        //
        return (nota1 + nota2) / 2;
    }

    2    static String verificarSituacao(double media) {                //
        if (media >= 7) {
            return "APROVADO";
        }
        if (media >= 5) {
            return "RECUPERAÇÃO";
        }
        return "REPROVADO";
    }

    static void apresentarResultado(String aluno, double media, String
situacao) { // 3
        System.out.println("=====");
        System.out.println("Aluno: " + aluno);
        System.out.printf("Média: %.2f%n", media);
        System.out.println("Situação: " + situacao);
        System.out.println("=====");
    }

    4    public static void main(String[] args) {                        //

        Scanner scanner = new Scanner(System.in);

        System.out.println("Nome do aluno:");
        String aluno = scanner.nextLine();

        System.out.println("Primeira nota:");
        double nota1 = Double.parseDouble(scanner.nextLine());

        System.out.println("Segunda nota:");
        double nota2 = Double.parseDouble(scanner.nextLine());

        5    double media = calcularMedia(nota1, nota2);                //
        6    String situacao = verificarSituacao(media);                //
        7    apresentarResultado(aluno, media, situacao);                //
    }
}

```

```
        scanner.close();  
    }  
}
```

Linha 1: declara um método que recebe duas notas e devolve a média. Ele não apresenta nada na tela — apenas calcula e devolve.

Linha 2: declara um método que recebe uma média e devolve a situação em texto. Repare que ele tem três comandos `return`: assim que um deles executa, o método termina imediatamente. Por isso não foi necessário usar `else`.

Linha 3: declara um método `void`, que apenas apresenta informações. Ele não devolve nada porque a tarefa dele é comunicar, não calcular.

Linha 4: o método principal. Note que ele ficou curto e legível: lê os dados e coordena as chamadas.

Linha 5: chama `calcularMedia` passando as duas notas e guarda o retorno.

Linha 6: chama `verificarSituacao` passando a média calculada.

Linha 7: chama `apresentarResultado` passando as três informações.

Compare mentalmente este `main` com um programa em que tudo estivesse escrito de uma vez. O comportamento é idêntico, mas a leitura é completamente diferente: aqui você entende o programa inteiro lendo sete linhas.

De olho nos detalhes

Cada método deve fazer uma coisa só

Esta é a orientação mais valiosa da lição. Repare na divisão do exemplo:

- `calcularMedia` só calcula;
- `verificarSituacao` só decide;
- `apresentarResultado` só apresenta.

Se `calcularMedia` também imprimisse a média na tela, ele não poderia ser reaproveitado em um programa que grava o resultado em arquivo. Manter cálculo e apresentação separados é o que torna o código reutilizável — e é a mesma ideia que vai organizar a aplicação inteira da lição 15.

Escopo: variáveis não atravessam métodos

```
static void exemplo() {  
    int x = 10;  
}  
  
static void outro() {  
    System.out.println(x);    // ERRO: x não existe aqui  
}
```

Variáveis declaradas dentro de um método existem apenas ali. Para compartilhar informação entre métodos, passe por parâmetro e devolva com `return`. É mais trabalhoso do que usar uma variável global, e é exatamente por isso que funciona melhor: você sempre sabe de onde veio cada valor.

Todo caminho precisa retornar

```
static String classificar(int nota) {  
    if (nota >= 7) {  
        return "Aprovado";  
    }  
    // ERRO de compilação: missing return statement  
}
```

Se o método declara um tipo de retorno, **todos os caminhos possíveis** precisam devolver algo. Em outras linguagens o retorno ausente vira um valor vazio e o problema só aparece muito depois; em Java, aparece na compilação.

Sobrecarga de métodos

Java permite vários métodos com o mesmo nome, desde que os parâmetros sejam diferentes:

```
static double calcularMedia(double n1, double n2) {  
    return (n1 + n2) / 2;  
}  
  
static double calcularMedia(double n1, double n2, double n3) {  
    return (n1 + n2 + n3) / 3;  
}
```

O compilador escolhe a versão correta observando os argumentos da chamada. Isso se chama sobrecarga, e é muito usado na biblioteca do Java — `System.out.println` tem mais de dez versões, uma para cada tipo que ele aceita.

Exercícios resolvidos

1) Crie um método **somar** que receba dois inteiros e devolva a soma.

```
public class Exercicio01 {  
  
    static int somar(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(somar(10, 5));    // 15  
        System.out.println(somar(100, 42)); // 142  
    }  
}
```

Repare que, uma vez escrito, o método pode ser chamado quantas vezes você quiser. Esse é o ganho.

2) Crie um método **ehPar** que receba um inteiro e devolva **true** ou **false**.

```
public class Exercicio02 {  
  
    static boolean ehPar(int numero) {  
        return numero % 2 == 0;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(ehPar(10)); // true  
        System.out.println(ehPar(7));  // false  
    }  
}
```

Não é preciso escrever `if (numero % 2 == 0) { return true; } else { return false; }`. A comparação já produz um boolean — basta devolvê-lo.

3) Crie um método **aplicarDesconto** que receba o valor e o percentual e devolva o valor com desconto.

```
import java.util.Scanner;  
  
public class Exercicio03 {  
  
    static double aplicarDesconto(double valor, double percentual) {  
        double desconto = valor * (percentual / 100);  
        return valor - desconto;  
    }  
  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.println("Digite o valor da compra:");  
        double valor = Double.parseDouble(scanner.nextLine());  
  
        System.out.println("Digite o percentual de desconto:");  
        double percentual = Double.parseDouble(scanner.nextLine());  
  
        double valorFinal = aplicarDesconto(valor, percentual);  
  
        System.out.printf("Valor final: R$ %.2f%n", valorFinal);  
  
        scanner.close();  
    }  
}
```

4) Crie um método **maiorDeTres** que receba três números e devolva o maior.

```
public class Exercicio04 {
```

```

static int maiorDeTres(int a, int b, int c) {
    int maior = a;

    if (b > maior) {
        maior = b;
    }
    if (c > maior) {
        maior = c;
    }

    return maior;
}

public static void main(String[] args) {
    System.out.println(maiorDeTres(10, 45, 32));    // 45
    System.out.println(maiorDeTres(80, 12, 7));    // 80
}
}

```

Compare com o exercício 5 da lição 8. A lógica é a mesma, mas agora ela virou uma ferramenta reutilizável.

5) Crie um método calcularIMC e um método classificarIMC, e use os dois juntos.

```

import java.util.Scanner;

public class Exercicio05 {

    static double calcularIMC(double peso, double altura) {
        return peso / (altura * altura);
    }

    static String classificarIMC(double imc) {
        if (imc < 18.5) {
            return "Abaixo do peso";
        }
        if (imc < 25) {
            return "Peso normal";
        }
        if (imc < 30) {
            return "Sobrepeso";
        }
        return "Obesidade";
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o seu peso em kg:");
        double peso = Double.parseDouble(scanner.nextLine());
    }
}

```

```

        System.out.println("Digite a sua altura em metros:");
        double altura = Double.parseDouble(scanner.nextLine());

        double imc = calcularIMC(peso, altura);

        System.out.printf("IMC: %.2f%n", imc);
        System.out.println("Classificação: " + classificarIMC(imc));

        scanner.close();
    }
}

```

Dois métodos independentes, cada um com uma responsabilidade, combinados no main. Este é o padrão que você deve buscar em todos os programas daqui em diante.

Exercícios

1. Crie um método `calcularArea` que receba base e altura e devolva a área de um retângulo.
2. Crie um método `converterCelsius` que receba uma temperatura em Celsius e devolva em Fahrenheit.
3. Crie um método void chamado `apresentarCabecalho` que receba um título e o imprima entre duas linhas de asteriscos.
4. Crie um método `ehMaiorDeIdade` que receba a idade e devolva um boolean, e use o retorno dentro de um `if` no main.
5. Crie um método `calcularParcela` que receba o valor total e o número de parcelas e devolva o valor de cada parcela.
6. Sobrecarregue um método `calcularMedia` para funcionar com duas e com três notas.
7. Crie um método `dobrar` que receba um número e devolva o dobro, e chame-o três vezes com valores diferentes.
8. Crie um método `saudacao` que receba um nome e devolva uma mensagem de boas-vindas personalizada.
9. Crie um método `calcularSalarioLiquido` que receba o salário bruto e o percentual de desconto e devolva o líquido.
10. Crie um método `menorDeTres` que receba três números e devolva o menor.
11. Reescreva o exercício 6 da lição 8 (cálculo de imposto) transformando o cálculo em um método.
12. Crie um método `validarSenha` que receba uma senha e devolva `true` se ela tiver mais de 5 caracteres.
13. Crie um programa com três métodos — um que lê os dados, um que processa e um que apresenta — e um main com apenas três linhas de chamada.
14. Crie um método `calcularCustoViagem` que receba distância, consumo do carro e preço do combustível e devolva o custo total.

Dica do dia

Se você precisa de um comentário para explicar um trecho do seu programa, provavelmente esse trecho deveria ser um método. O nome do método explica sozinho o que ele faz — e, diferente do comentário, ele nunca fica desatualizado.

DIA 11 — MÉTODOS PARA MANIPULAÇÃO DE TEXTO

Objetivo do dia

Criar programas que realizam diferentes tipos de manipulação de texto.

Introdução

Na aula anterior você aprendeu a criar os seus próprios métodos.

Na aula de hoje você vai aprender a usar os métodos que o Java já traz prontos para trabalhar com texto. São dezenas deles, e eles resolvem quase tudo que você vai precisar fazer com um String.

A característica mais importante de String

Antes de qualquer método, você precisa saber disto: **em Java, textos são imutáveis**. Nenhum método altera o texto original; todos devolvem um texto novo.

```
String nome = "maria";

nome.toUpperCase();
System.out.println(nome);    // maria — nada mudou!

nome = nome.toUpperCase();
System.out.println(nome);    // MARIA — agora sim
```

Esquecer de guardar o retorno é o erro número um desta lição. Sempre que usar um método de texto, pergunte-se: **onde estou guardando o resultado?**

Os métodos que você mais vai usar

Método	O que faz	Exemplo	Resultado
length()	devolve o tamanho	"Java".length()	4
toUpperCase()	converte para maiúsculas	"java".toUpperCase()	"JAVA"
toLowerCase()	converte para minúsculas	"JAVA".toLowerCase()	"java"
strip()	remove espaços das pontas	" java".strip()	"java"

Método	O que faz	Exemplo	Resultado
<code>substring(i)</code>	do índice até o final	<code>"Java".substring(1)</code>	<code>"ava"</code>
<code>substring(i, f)</code>	do índice i até f - 1	<code>"Java".substring(0, 2)</code>	<code>"Ja"</code>
<code>charAt(i)</code>	o caractere na posição	<code>"Java".charAt(0)</code>	<code>'J'</code>
<code>indexOf(t)</code>	posição do trecho, ou -1	<code>"Java".indexOf("v")</code>	<code>2</code>
<code>contains(t)</code>	contém o trecho?	<code>"Java".contains("av")</code>	<code>true</code>
<code>startsWith(t)</code>	começa com?	<code>"Java".startsWith("Ja")</code>	<code>true</code>
<code>endsWith(t)</code>	termina com?	<code>"Java".endsWith("va")</code>	<code>true</code>
<code>replace(a, b)</code>	troca ocorrências	<code>"Java".replace("a", "o")</code>	<code>"Jovo"</code>
<code>split(sep)</code>	divide em várias partes	<code>"a,b,c".split(",")</code>	<code>["a", "b", "c"]</code>
<code>isEmpty()</code>	está vazio?	<code>"".isEmpty()</code>	<code>true</code>
<code>isBlank()</code>	vazio ou só espaços?	<code>" ".isBlank()</code>	<code>true</code>
<code>equals(t)</code>	tem o mesmo conteúdo?	<code>"a".equals("a")</code>	<code>true</code>
<code>equalsIgnoreCase(t)</code>	igual ignorando a caixa	<code>"A".equalsIgnoreCase("a")</code>	<code>true</code>
<code>repeat(n)</code>	repete o texto n vezes	<code>"ab".repeat(3)</code>	<code>"ababab"</code>
<code>String.join(s, . .)</code>	junta com um separador	<code>String.join("-", "a", "b")</code>	<code>"a-b"</code>

Você não precisa decorar essa tabela. Precisa saber que ela existe e consultá-la. Com o tempo, os dez primeiros métodos vão para a memória sozinhos.

Índices começam em zero

```
String texto = "JAVA";
//           0123

System.out.println(texto.charAt(0)); // J
System.out.println(texto.charAt(3)); // A
System.out.println(texto.length()); // 4
System.out.println(texto.substring(0, 2)); // JA
```

Duas observações importantes:

O último índice é sempre `length()` - 1. O texto "JAVA" tem tamanho 4 e índices de 0 a 3. Tentar acessar `charAt(4)` provoca erro em execução.

No `substring(início, fim)`, o início é incluído e o fim é excluído. Por isso `substring(0, 2)` devolve os caracteres das posições 0 e 1.

Montando textos com valores de variáveis

Existem três formas de combinar texto e variáveis. Conheça as três e escolha conforme o caso.

1. Concatenação com o sinal +

É a forma direta, ótima para mensagens curtas:

```
String nome = "Maria";
String curso = "Imersão Core Dev";

System.out.println("Olá " + nome + ", você está na " + curso + "!!!");
```

2. String.format e printf

Melhor quando há números para formatar:

```
String mensagem = String.format("Olá %s, você está na %s!!!", nome,
curso);
System.out.println(mensagem);

System.out.printf("Total: R$ %.2f%n", 1234.5); // Total: R$ 1234,50
```

A diferença entre os dois: `String.format` **devolve** o texto formatado, enquanto `printf` já **apresenta** direto no console.

3. Text block

A partir do Java 15, para textos de várias linhas, existe o bloco delimitado por três aspas duplas:

```
String menu = """
=====
        MENU PRINCIPAL
=====
1 - Cadastrar
2 - Listar
3 - Sair
===== """;
```

```
System.out.println(menu);
```

O texto é apresentado exatamente como foi escrito, sem precisar de `\n` nem de concatenação. Ele não substitui variáveis sozinho — para isso, combine com `String.format`.

Vamos analisar o código

Crie o arquivo `ManipulacaoTexto.java`:

```

import java.util.Scanner;

public class ManipulacaoTexto {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o seu nome completo:");
        String nomeCompleto = scanner.nextLine().strip(); // 1

        System.out.println("Maiúsculas: " +
            nomeCompleto.toUpperCase()); // 2
        System.out.println("Minúsculas: " +
            nomeCompleto.toLowerCase());
        System.out.println("Quantidade de caracteres: " +
            nomeCompleto.length()); // 3
        System.out.println("Primeira letra: " +
            nomeCompleto.charAt(0)); // 4

        String[] partes = nomeCompleto.split(" "); // 5
        String primeiroNome = partes[0]; // 6
        String ultimoSobrenome = partes[partes.length - 1]; // 7

        System.out.println("Primeiro nome: " + primeiroNome);
        System.out.println("Último sobrenome: " + ultimoSobrenome);
        System.out.println("Quantidade de palavras: " +
            partes.length);

        System.out.printf("Olá %s, seja bem-vindo!\n", primeiroNome); // 8

        scanner.close();
    }
}

```

Linha 1: lê o nome digitado e já remove os espaços das pontas com `strip()`. Repare que os dois métodos foram encadeados: primeiro `nextLine()` devolve o texto, depois `strip()` age sobre esse texto. Esse encadeamento é muito comum em Java.

Linha 2: apresenta o nome em maiúsculas. Note que a variável `nomeCompleto` **não foi alterada** — apenas usamos o valor devolvido.

Linha 3: apresenta a quantidade de caracteres. Atenção: `length()` com parênteses, porque é um método.

Linha 4: apresenta o caractere da posição zero, ou seja, a primeira letra.

Linha 5: divide o texto em várias partes, usando o espaço como separador. O resultado é um array de textos — a estrutura que você vai estudar em detalhe amanhã.

Linha 6: pega o primeiro elemento do array, que está na posição zero.

Linha 7: pega o último elemento. Como os índices vão de 0 até `length - 1`, essa é a fórmula para o último. Repare que aqui `length` está **sem** parênteses: em arrays, `length` é uma propriedade, não um método. Essa diferença confunde muita gente no começo.

Linha 8: apresenta a mensagem final com `printf`, usando `%s` como marcador de texto.

De olho nos detalhes

O erro que interrompe o programa

```
String nome = null;  
System.out.println(nome.toUpperCase()); // NullPointerException
```

Chamar um método sobre uma variável que vale `null` encerra o programa com erro. Antes de manipular um texto que pode não ter vindo, verifique:

```
if (nome != null && !nome.isBlank()) {  
    System.out.println(nome.toUpperCase());  
}
```

Existe também um truque simples e muito usado: ao comparar com um valor fixo, coloque o valor fixo à esquerda.

```
"admin".equals(nome) // seguro mesmo se nome for null  
nome.equals("admin") // quebra se nome for null
```

`length()` ou `length`?

- Em `String`: `nome.length()` — método, com parênteses.
- Em array: `partes.length` — propriedade, sem parênteses.

Não há lógica profunda por trás disso; é herança histórica da linguagem. Erre algumas vezes e depois estará memorizado.

Não existe crase em Java

Se você já viu textos delimitados por crase em outra linguagem, esqueça esse recurso aqui. As três formas de montar texto em Java são as que você acabou de aprender.

O `split` aceita mais de um caractere

```
String data = "04/08/2026";  
String[] partes = data.split("/");  
  
System.out.println("Dia: " + partes[0]);  
System.out.println("Mês: " + partes[1]);  
System.out.println("Ano: " + partes[2]);
```

Exercícios resolvidos

1) Crie um programa que leia um nome completo e apresente apenas o primeiro nome.

```
import java.util.Scanner;

public class Exercicio01 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o seu nome completo:");
        String nomeCompleto = scanner.nextLine().strip();

        String primeiroNome = nomeCompleto.split(" ")[0];

        System.out.println("Primeiro nome: " + primeiroNome);

        scanner.close();
    }
}
```

2) Crie um programa que leia um e-mail e verifique se ele é válido de forma simples.

```
import java.util.Scanner;

public class Exercicio02 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o seu e-mail:");
        String email = scanner.nextLine().strip().toLowerCase();

        boolean valido = email.contains("@") && email.contains(".") &&
email.length() > 5;

        if (valido) {
            System.out.println("E-mail válido.");
        } else {
            System.out.println("E-mail inválido.");
        }

        scanner.close();
    }
}
```

3) Crie um programa que leia um texto e apresente-o invertido.

```
import java.util.Scanner;

public class Exercicio03 {
```

```

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.println("Digite um texto:");
    String texto = scanner.nextLine();

    String invertido = new
StringBuilder(texto).reverse().toString();

    System.out.println("Invertido: " + invertido);

    scanner.close();
}
}

```

O `StringBuilder` é uma classe feita para construir e modificar textos. Como `String` é imutável, quando precisamos alterar muito um texto, o `StringBuilder` é a ferramenta correta.

4) Crie um programa que leia um nome completo e apresente as iniciais de cada palavra.

```

import java.util.Scanner;

public class Exercicio04 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o seu nome completo:");
        String nome = scanner.nextLine().strip();

        String[] palavras = nome.split(" ");
        String iniciais = "";

        for (String palavra : palavras) {
            if (!palavra.isBlank()) {
                iniciais += palavra.toUpperCase().charAt(0) + ".";
            }
        }

        System.out.println("Iniciais: " + iniciais);

        scanner.close();
    }
}

```

Apareceu aqui um comando de repetição, que é assunto da lição 14. Por enquanto, leia o `for` como “para cada palavra dentro de `palavras`”. Repare também no `+=` acumulando o texto — exatamente o que você aprendeu na lição 6.

5) Crie um programa que leia uma palavra e informe se ela é um palíndromo.

```
import java.util.Scanner;

public class Exercicio05 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite uma palavra:");
        String palavra = scanner.nextLine().strip().toLowerCase();

        String invertida = new
        StringBuilder(palavra).reverse().toString();

        if (palavra.equals(invertida)) {
            System.out.println("É um palíndromo!");
        } else {
            System.out.println("Não é um palíndromo.");
        }

        scanner.close();
    }
}
```

Repare no `toLowerCase()` antes da comparação. Sem ele, “Arara” não seria reconhecida, porque o “A” maiúsculo é diferente do “a” minúsculo.

Exercícios

1. Crie um programa que leia um nome e apresente-o em maiúsculas e em minúsculas.
2. Crie um programa que leia um texto e apresente a quantidade de caracteres.
3. Crie um programa que leia um nome completo e apresente o último sobrenome.
4. Crie um programa que leia uma frase e conte quantas palavras ela tem.
5. Crie um programa que leia um CEP no formato 89000-000 e apresente apenas os números.
6. Crie um programa que leia um nome de usuário e valide se ele tem no mínimo 5 caracteres e não contém espaços.
7. Crie um programa que leia uma data no formato 04/08/2026 e apresente dia, mês e ano em linhas separadas.
8. Crie um programa que leia um texto e substitua todos os espaços por hífen.
9. Crie um programa que leia um nome e apresente-o com a primeira letra maiúscula e o restante em minúsculas.
10. Crie um programa que leia um telefone com máscara e apresente apenas os dígitos.
11. Crie um programa que leia duas palavras e informe se a primeira contém a segunda.
12. Crie um programa que leia um texto e apresente a primeira e a última letra.
13. Crie um programa que apresente um cabeçalho de menu usando text block.

14. Crie um programa que leia um nome e apresente uma linha de traços do mesmo tamanho do nome (dica: use `repeat`).

Dica do dia

Textos em Java são imutáveis. Nenhum método altera o texto original: todos devolvem um novo. Se o seu programa “não muda o texto”, verifique se você guardou o valor devolvido em algum lugar.

DIA 12 — COMO TRABALHAR COM LISTAS

Objetivo do dia

Criar programas que armazenam e manipulam várias informações dentro de uma única estrutura.

Introdução

Até agora, cada informação exigia uma variável própria. Para guardar três nomes você criou três variáveis. Para guardar trezentos, esse caminho é impraticável.

Na aula de hoje você vai aprender a guardar várias informações em uma única estrutura.

Java tem duas estruturas de lista

Esta é a primeira decisão que você precisa saber tomar:

- **Array:** tamanho fixo, definido na criação. Rápido e simples.
- **ArrayList:** tamanho dinâmico, cresce e diminui conforme você adiciona e remove.

Na prática, você vai usar `ArrayList` em quase tudo. Mas o `array` aparece o tempo todo — inclusive no `String[] args` do seu `main` e no retorno do `split()` da lição passada — então precisamos conhecer os dois.

Array: a lista de tamanho fixo

```
String[] frutas = {"Mamão", "Abacaxi", "Laranja", "Banana"};
```

```
System.out.println(frutas[0]);           // Mamão
System.out.println(frutas[3]);           // Banana
System.out.println(frutas.length);       // 4
```

```
frutas[1] = "Manga";                      // altera a posição 1
```

Para criar um `array` vazio, é preciso informar o tamanho de antemão:

```
String[] nomes = new String[5];          // 5 posições, todas valendo null
int[] numeros = new int[3];              // 3 posições, todas valendo 0
```

E esse tamanho **não muda mais**. Não existe forma de acrescentar uma sexta posição a um array de cinco.

ArrayList: a lista de tamanho dinâmico

```
import java.util.ArrayList;
import java.util.List;

List<String> frutas = new ArrayList<>();

frutas.add("Mamão");
frutas.add("Abacaxi");
frutas.add("Laranja");

System.out.println(frutas.get(0)); // Mamão
System.out.println(frutas.size()); // 3
System.out.println(frutas);        // [Mamão, Abacaxi, Laranja]
```

Vamos abrir a linha de criação, porque ela tem várias novidades:

```
List<String> frutas = new ArrayList<>();
//      |           |           |           |
//      |           |           |           | a implementação concreta
//      |           |           | nome da variável
//      |           | o tipo dos elementos (generics)
//      | o tipo da variável
```

O `<String>` entre os sinais de menor e maior é o que chamamos de **generics**: ele informa que essa lista guarda textos, e apenas textos. Se você tentar adicionar um número, o código não compila.

Repare também que declaramos a variável como `List` e criamos um `ArrayList`. Isso é uma boa prática consagrada: você programa pensando na lista, não na implementação específica. Por enquanto, apenas siga o padrão; o motivo completo aparece no estudo de orientação a objetos.

Os principais métodos de lista

Método	O que faz
<code>add(x)</code>	adiciona no final
<code>add(i, x)</code>	insere na posição <code>i</code>
<code>get(i)</code>	devolve o elemento da posição <code>i</code>
<code>set(i, x)</code>	substitui o elemento da posição <code>i</code>
<code>remove(i)</code>	remove o elemento da posição <code>i</code>
<code>size()</code>	devolve a quantidade de elementos
<code>contains(x)</code>	verifica se o elemento existe
<code>indexOf(x)</code>	devolve a posição do elemento, ou <code>-1</code>
<code>isEmpty()</code>	verifica se a lista está vazia

Método	O que faz
<code>clear()</code>	remove todos os elementos

E dois utilitários muito úteis:

```
String.join(", ", frutas);           // junta tudo em um texto único
Collections.sort(frutas);           // ordena a lista (precisa importar
java.util.Collections)
```

Percorrendo uma lista

A forma mais limpa é o `for` aprimorado, também chamado de *for-each*:

```
for (String fruta : frutas) {
    System.out.println(fruta);
}
```

Leia como: “para cada fruta dentro de `frutas`, faça...”.

Quando você também precisa da posição, use o `for` tradicional:

```
for (int i = 0; i < frutas.size(); i++) {
    System.out.println((i + 1) + " - " + frutas.get(i));
}
```

Os dois comandos serão detalhados na lição 14. Por enquanto, use-os como receita.

Vamos analisar o código

Crie o arquivo `ListaCompras.java`:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class ListaCompras {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        List<String> compras = new ArrayList<>();           // 1

        System.out.println("Digite 3 produtos:");
        compras.add(scanner.nextLine());                   // 2
        compras.add(scanner.nextLine());
        compras.add(scanner.nextLine());

        System.out.println("--- LISTA DE COMPRAS ---");
        for (int i = 0; i < compras.size(); i++) {          // 3
            System.out.println((i + 1) + " - " + compras.get(i));
        }
    }
}
```

```

4      System.out.println("Total de itens: " + compras.size());    //
5      System.out.println("Primeiro item: " + compras.get(0));      //
      System.out.println("Último item: " +
compras.get(compras.size() - 1)); // 6

      Collections.sort(compras);                                    // 7
      System.out.println("Em ordem alfabética: " + compras);

      compras.remove(0);                                            // 8
      System.out.println("Após remover o primeiro: " + compras);

      System.out.println("Contém 'Arroz'? " +
compras.contains("Arroz")); // 9

      scanner.close();
    }
}

```

Linha 1: cria uma lista vazia que vai guardar textos.

Linha 2: lê o produto digitado e adiciona ao final da lista. Repare que não precisamos dizer em que posição — o add sempre acrescenta no fim.

Linha 3: percorre a lista usando o índice, para poder apresentar a numeração. Note o `(i + 1)`: como os índices começam em zero, somamos 1 para que o usuário veja a contagem começando em 1.

Linha 4: apresenta a quantidade de elementos com `size()`.

Linha 5: pega o primeiro elemento, que está sempre na posição 0.

Linha 6: pega o último elemento. A fórmula `size() - 1` sempre aponta para a última posição.

Linha 7: ordena a lista em ordem alfabética. O método altera a própria lista, por isso não guardamos retorno.

Linha 8: remove o elemento da posição 0. Todos os outros são deslocados uma posição para trás automaticamente.

Linha 9: verifica se um valor existe na lista. Repare que `contains` compara conteúdo, e não referência — ele já usa `equals` internamente.

De olho nos detalhes

O erro de índice inválido

```

List<String> nomes = new ArrayList<>();
nomes.add("Maria");

```

```
System.out.println(nomes.get(1)); // IndexOutOfBoundsException
```

Uma lista com 1 elemento tem apenas o índice 0. Os índices sempre vão de 0 até `size() - 1`.

Diferente de outras linguagens, que devolvem um valor vazio quando você acessa uma posição inexistente, o Java interrompe o programa imediatamente. Isso é bom: você descobre o erro no momento exato em que ele acontece.

Cuidado especial ao remover números

```
List<Integer> numeros = new ArrayList<>(List.of(10, 20, 30));
```

```
numeros.remove(1); // remove a POSIÇÃO 1 → sobra [10, 30]
numeros.remove(Integer.valueOf(10)); // remove o VALOR 10
```

Quando a lista é de números inteiros, `remove(1)` é ambíguo para o programador, mas não para o compilador: ele entende como posição. Para remover pelo valor, use `Integer.valueOf`.

List.of cria uma lista imutável

```
List<String> fixa = List.of("a", "b", "c");
fixa.add("d"); // UnsupportedOperationException em execução
```

`List.of` é ótimo para dados que não mudam. Se você precisa adicionar ou remover depois, envolva em um `ArrayList`:

```
List<String> lista = new ArrayList<>(List.of("a", "b", "c"));
lista.add("d"); // agora funciona
```

Listas guardam objetos, não primitivos

```
List<int> numeros; // ERRO
List<Integer> numeros; // correto
```

Para guardar números em uma lista, use as classes equivalentes: `Integer` para `int`, `Double` para `double`, `Boolean` para `boolean`. O Java faz a conversão automaticamente quando você adiciona ou lê valores, então na prática você quase não percebe a diferença.

Exercícios resolvidos

1) Crie uma lista com 5 nomes e apresente todos.

```
import java.util.List;

public class Exercicio01 {
    public static void main(String[] args) {
        List<String> nomes = List.of("Ana", "Bruno", "Carla", "Diego",
        "Elisa");

        for (String nome : nomes) {
            System.out.println(nome);
        }
    }
}
```

```
}  
}
```

2) Crie uma lista de números e apresente a soma de todos.

```
import java.util.List;  
  
public class Exercicio02 {  
    public static void main(String[] args) {  
        List<Integer> numeros = List.of(10, 25, 3, 47, 8);  
  
        int soma = 0;  
  
        for (int numero : numeros) {  
            soma += numero;  
        }  
  
        System.out.println("Soma: " + soma);  
    }  
}
```

Repare no acumulador que você aprendeu na lição 6: soma começa em zero e vai crescendo a cada volta.

3) Crie uma lista de números e apresente o maior e o menor.

```
import java.util.List;  
  
public class Exercicio03 {  
    public static void main(String[] args) {  
        List<Integer> numeros = List.of(10, 25, 3, 47, 8);  
  
        int maior = numeros.get(0);  
        int menor = numeros.get(0);  
  
        for (int numero : numeros) {  
            if (numero > maior) {  
                maior = numero;  
            }  
            if (numero < menor) {  
                menor = numero;  
            }  
        }  
  
        System.out.println("Maior: " + maior);  
        System.out.println("Menor: " + menor);  
    }  
}
```

Detalhe importante: iniciei maior e menor com o primeiro elemento da lista, e não com zero. Se eu tivesse usado zero, o programa daria resposta errada em uma lista contendo apenas números negativos.

4) Crie uma lista de nomes e apresente todos em uma única linha, separados por vírgula.

```
import java.util.List;

public class Exercicio04 {
    public static void main(String[] args) {
        List<String> nomes = List.of("Ana", "Bruno", "Carla");

        System.out.println(String.join(", ", nomes));    // Ana, Bruno,
Carla
    }
}
```

5) Crie um programa que leia produtos até o usuário digitar “fim” e apresente a lista completa.

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Exercicio05 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> produtos = new ArrayList<>();

        System.out.println("Digite os produtos (ou 'fim' para
encerrar):");

        while (true) {
            String produto = scanner.nextLine().strip();

            if (produto.equalsIgnoreCase("fim")) {
                break;
            }

            produtos.add(produto);
        }

        System.out.println("--- PRODUTOS CADASTRADOS ---");
        for (int i = 0; i < produtos.size(); i++) {
            System.out.println((i + 1) + " - " + produtos.get(i));
        }

        System.out.println("Total: " + produtos.size() + "
produto(s)");
    }
}
```

```
        scanner.close();  
    }  
}
```

Este exercício usa o comando `while`, que é o assunto de amanhã. Aproveite para observá-lo funcionando antes de estudá-lo: o programa repete a leitura até o usuário digitar “fim”.

Exercícios

1. Crie uma lista vazia, leia 3 produtos do teclado, adicione à lista e apresente-os numerados.
2. Crie uma lista de preços e apresente a média.
3. Crie uma lista de nomes e verifique se um nome digitado pelo usuário existe nela.
4. Crie uma lista de números e apresente apenas os pares.
5. Crie uma lista de nomes, remova o segundo elemento e apresente o resultado.
6. Crie duas listas e apresente uma terceira contendo os elementos das duas.
7. Crie uma lista de números e conte quantos são maiores que 10.
8. Crie uma lista de nomes e apresente-a em ordem alfabética.
9. Crie uma lista de nomes e apresente-a em ordem alfabética inversa (dica: pesquise `Collections.reverse()`).
10. Crie uma lista de produtos e permita ao usuário remover um item informando a posição.
11. Crie uma lista de notas, calcule a média e apresente quantas notas estão acima dela.
12. Crie uma lista de nomes e apresente apenas os que começam com a letra “A”.
13. Crie um array de 5 posições, preencha com números e apresente a soma.
14. Leia uma frase e use `split` para criar uma lista com as palavras, apresentando quantas palavras existem.

Dica do dia

Índices sempre começam em zero e terminam em `size()` - 1. Quase todo erro de `IndexOutOfBoundsException` vem de esquecer isso. Quando percorrer uma lista com índice, a condição correta é `i < lista.size()`, nunca `i <= lista.size()`.

DIA 13 — LAÇOS DE REPETIÇÃO COM O COMANDO WHILE

Objetivo do dia

Criar programas que repetem um bloco de comandos enquanto uma condição for verdadeira.

Introdução

Na aula anterior você aprendeu a guardar várias informações em listas.

Na aula de hoje você vai aprender a repetir comandos. Este é o recurso que transforma programas pequenos em programas úteis: em vez de escrever a mesma instrução dez vezes, você escreve uma vez e manda repetir.

O comando while

```
while (condicao) {  
    // repete enquanto a condição for verdadeira  
}
```

O funcionamento é simples: a condição é testada, e se for verdadeira o bloco executa. Ao final do bloco, a condição é testada de novo. E de novo. Até que ela se torne falsa.

```
int contador = 1;  
  
while (contador <= 5) {  
    System.out.println(contador);  
    contador++;  
}
```

Saída:

```
1  
2  
3  
4  
5
```

As três partes de todo laço

Olhe de novo para o exemplo e identifique três elementos. Eles sempre existem, em todo laço de repetição que você escrever:

```
int contador = 1;           // 1. INICIALIZAÇÃO  
while (contador <= 5) {    // 2. CONDIÇÃO DE PARADA  
    System.out.println(contador);  
    contador++;            // 3. ATUALIZAÇÃO  
}
```

Inicialização — a variável de controle recebe um valor inicial, antes do laço.

Condição de parada — a pergunta que define se o laço continua. Enquanto verdadeira, repete.

Atualização — a mudança na variável de controle que, eventualmente, torna a condição falsa.

Se qualquer uma das três faltar, o laço não funciona. E a que mais falta é a terceira.

O laço infinito

```
int contador = 1;  
  
while (contador <= 5) {  
    System.out.println(contador);  
}
```

```
    // esqueceu o contador++  
}
```

Este programa apresenta o número 1 para sempre. A condição nunca se torna falsa, porque nada dentro do bloco altera o contador.

Se isso acontecer com você, pressione **Ctrl + C** no terminal para interromper a execução.

Todo programador escreve um laço infinito na primeira semana. Não se assuste: o importante é reconhecer o sintoma — o programa não para, ou repete sempre a mesma coisa — e ir direto olhar a linha de atualização.

O comando do/while

Existe uma variação em que a condição é testada **depois** da execução, garantindo que o bloco execute pelo menos uma vez:

```
do {  
    // executa pelo menos uma vez  
} while (condicao);
```

Repare no ponto e vírgula ao final. Ele é obrigatório nesta forma.

Esta variação é perfeita para menus:

```
int opcao;  
  
do {  
    System.out.println("1 - Cadastrar");  
    System.out.println("2 - Listar");  
    System.out.println("0 - Sair");  
    System.out.print("Escolha uma opção: ");  
  
    opcao = Integer.parseInt(scanner.nextLine());  
  
    switch (opcao) {  
        case 1 -> System.out.println("Cadastrando...");  
        case 2 -> System.out.println("Listando...");  
        case 0 -> System.out.println("Encerrando...");  
        default -> System.out.println("Opção inválida.");  
    }  
} while (opcao != 0);
```

Faz sentido que o menu apareça pelo menos uma vez antes de qualquer verificação — por isso o do/while é a escolha natural aqui. Guarde este trecho: ele é o coração da aplicação da lição 15.

Os comandos break e continue

```
while (true) {  
    String comando = scanner.nextLine();  
  
    if (comando.equalsIgnoreCase("sair")) {
```

```

        break;           // encerra o laço imediatamente
    }

    if (comando.isBlank()) {
        continue;        // pula para a próxima repetição
    }

    System.out.println("Comando recebido: " + comando);
}

```

- **break** encerra o laço na hora, ignorando o restante do bloco e a condição.
- **continue** interrompe apenas a volta atual e vai direto para o próximo teste da condição.

O `while (true)` cria um laço que nunca termina sozinho — e por isso obriga a existência de um `break` em algum lugar. É um recurso legítimo e muito usado, mas prefira, quando possível, uma condição que descreva a regra real.

Vamos analisar o código

Crie o arquivo `Somatorio.java`:

```

import java.util.Scanner;

public class Somatorio {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double soma = 0;           // 1
        int quantidade = 0;         // 2
        boolean continuar = true;  // 3

        while (continuar) {        // 4
            System.out.println("Digite um número (ou 'fim' para
encerrar):");
            String entrada = scanner.nextLine().strip();

            if (entrada.equalsIgnoreCase("fim")) { // 5
                continuar = false;
            } else {
                soma += Double.parseDouble(entrada); // 6
                quantidade++; // 7
            }
        }

        System.out.println("Quantidade de números: " + quantidade);
        System.out.printf("Soma: %.2f%n", soma);

        if (quantidade > 0) { // 8
            System.out.printf("Média: %.2f%n", soma / quantidade);
        }
    }
}

```

```

    } else {
        System.out.println("Nenhum número foi informado.");
    }

    scanner.close();
}
}

```

Linha 1: o acumulador, iniciado em zero.

Linha 2: o contador, iniciado em zero.

Linha 3: a variável de controle do laço. Usar um `boolean` com nome descritivo deixa a condição do `while` muito clara.

Linha 4: o laço repete enquanto `continuar` for verdadeiro. Note que `while (continuar)` já é uma condição booleana completa — não é preciso escrever `while (continuar == true)`.

Linha 5: verifica se o usuário digitou a palavra de encerramento. Em caso positivo, a variável de controle muda e o laço termina na próxima verificação.

Linha 6: converte o texto digitado e acumula na soma.

Linha 7: incrementa o contador.

Linha 8: esta verificação evita uma divisão por zero. Se o usuário digitasse “fim” logo na primeira pergunta, quantidade seria zero e o cálculo da média produziria um resultado inválido. Programas robustos sempre pensam no caso vazio.

De olho nos detalhes

while ou do/while?

Use **while** quando é possível que o bloco não precise executar nenhuma vez. Use **do/while** quando ele obrigatoriamente executa pelo menos uma vez, como em menus e validações de entrada.

Não altere a condição fora do laço por engano

Um erro comum é atualizar a variável de controle dentro de um `if` que nem sempre executa:

```

while (contador <= 5) {
    if (algumaCondicao) {
        contador++; // só incrementa às vezes → risco de laço
        infinito
    }
}

```

Se a atualização depende de uma condição, garanta que exista um caminho que sempre leve ao encerramento.

Cuidado com a divisão por zero

Como no exemplo acima: sempre que dividir por uma variável, pergunte-se se ela pode valer zero.

Exercícios resolvidos

1) Crie um programa que apresente os números de 1 a 10 usando while.

```
public class Exercicio01 {
    public static void main(String[] args) {
        int contador = 1;

        while (contador <= 10) {
            System.out.println(contador);
            contador++;
        }
    }
}
```

2) Crie um programa que apresente a tabuada de um número lido do teclado.

```
import java.util.Scanner;

public class Exercicio02 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite um número:");
        int numero = Integer.parseInt(scanner.nextLine());

        int multiplicador = 1;

        while (multiplicador <= 10) {
            System.out.println(numero + " x " + multiplicador + " = "
+ (numero * multiplicador));
            multiplicador++;
        }

        scanner.close();
    }
}
```

3) Crie um programa que peça uma senha até o usuário acertar.

```
import java.util.Scanner;

public class Exercicio03 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
}
```

```

String senha;
int tentativas = 0;

do {
    System.out.println("Digite a senha:");
    senha = scanner.nextLine();
    tentativas++;

    if (!senha.equals("1234")) {
        System.out.println("Senha incorreta. Tente novamente.");
    }
} while (!senha.equals("1234"));

System.out.println("Acesso liberado em " + tentativas + " tentativa(s).");

scanner.close();
}
}

```

O do/while é a escolha certa aqui: a senha precisa ser pedida pelo menos uma vez antes de qualquer verificação.

4) Crie um programa que leia um número e apresente o fatorial dele.

```

import java.util.Scanner;

public class Exercicio04 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite um número:");
        int numero = Integer.parseInt(scanner.nextLine());

        long fatorial = 1;
        int contador = numero;

        while (contador > 1) {
            fatorial *= contador;
            contador--;
        }

        System.out.println(numero + "! = " + fatorial);

        scanner.close();
    }
}

```

Note que o acumulador fatorial começa em 1, e não em zero: qualquer número multiplicado por zero resultaria em zero. Note também o tipo `long`: o fatorial cresce tão rápido que $13!$ já ultrapassa o limite do `int`.

5) Crie um menu que só encerra quando o usuário escolher a opção 0.

```
import java.util.Scanner;

public class Exercicio05 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int opcao;

        do {
            System.out.println("""
                =====
                        MENU PRINCIPAL
                =====
                1 - Consultar saldo
                2 - Depositar
                3 - Sacar
                0 - Sair
                =====""");
            System.out.print("Escolha uma opção: ");

            opcao = Integer.parseInt(scanner.nextLine());

            switch (opcao) {
                case 1 -> System.out.println("Seu saldo é R$
1.250,00");
                case 2 -> System.out.println("Abrindo a tela de
depósito...");
                case 3 -> System.out.println("Abrindo a tela de
saque...");
                case 0 -> System.out.println("Encerrando. Até logo!");
                default -> System.out.println("Opção inválida.");
            }

            System.out.println();
        } while (opcao != 0);

        scanner.close();
    }
}
```

Guarde bem este exercício. Ele é, na prática, o esqueleto da aplicação que vamos construir na lição 15.

Exercícios

1. Crie um programa que apresente os números de 10 a 1, em contagem regressiva.
2. Crie um programa que some os números de 1 a 100 e apresente o total.
3. Crie um programa que leia números até o usuário digitar 0 e apresente a soma de todos.
4. Crie um programa que apresente os números pares de 0 a 20.
5. Crie um programa que leia números até o usuário digitar “fim” e apresente o maior deles.
6. Crie um programa que leia uma idade e repita a pergunta enquanto o valor informado for negativo.
7. Crie um programa que apresente os múltiplos de 3 entre 1 e 50.
8. Crie um programa que leia um número e apresente a soma de todos os números de 1 até ele.
9. Crie um programa que leia notas até o usuário digitar -1 e apresente a média das notas válidas.
10. Crie um programa que simule um caixa: leia preços até o usuário digitar 0 e apresente o total da compra e a quantidade de itens.
11. Crie um programa que leia um número e apresente quantos dígitos ele tem (dica: divida por 10 até chegar a zero).
12. Crie um programa que leia um número e apresente a sequência de Fibonacci até esse valor.
13. Crie um menu com quatro opções que só encerra na opção 0, usando `do/while` e `switch`.
14. Crie um programa que sorteie um número de 1 a 100 com `Math.random()` e peça ao usuário que adivinhe, informando “maior” ou “menor” a cada tentativa.

O exercício 14 é o seu primeiro jogo. Vale o esforço.

Dica do dia

Todo laço precisa de três coisas: inicialização, condição de parada e atualização. Quando um programa repetir para sempre, procure primeiro a atualização que faltou. E lembre-se: **Ctrl + C** interrompe a execução no terminal.

DIA 14 — LAÇOS DE REPETIÇÃO COM O COMANDO FOR

Objetivo do dia

Criar programas que utilizam o comando `for` para construir laços de repetição.

Introdução

Na aula anterior você aprendeu a repetir comandos com o `while`.

Na aula de hoje você vai aprender um segundo comando de repetição. Ele faz exatamente a mesma coisa, mas organiza as três partes do laço em um único lugar — o que o torna a escolha ideal quando você já sabe quantas repetições precisa.

O comando for

```
for (inicializacao; condicao; atualizacao) {  
    // bloco repetido  
}
```

Compare os dois códigos abaixo. Eles fazem exatamente a mesma coisa:

```
// com while  
int i = 1;  
while (i <= 5) {  
    System.out.println(i);  
    i++;  
}  
  
// com for  
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

No for, as três partes ficam reunidas na primeira linha:

```
for (int i = 1; i <= 5; i++) {  
//      |           |           | atualização: executa ao final de cada  
volta  
//      |           |           | condição: testada antes de cada volta  
//      |           |           | inicialização: executa uma única vez
```

A ordem de execução é: inicializa uma vez, testa, executa o bloco, atualiza, testa de novo, e assim por diante.

Quando usar for e quando usar while

A regra prática:

- **for** quando você sabe quantas repetições serão feitas: percorrer uma lista, contar de 1 a 100, apresentar uma tabuada.
- **while** quando você não sabe: ler até o usuário digitar “fim”, tentar até acertar a senha.

Há também uma vantagem de escopo no for: a variável `i` declarada nele existe apenas dentro do laço, o que evita reaproveitamentos acidentais.

O for aprimorado (for-each)

Para percorrer listas e arrays sem se preocupar com índices:

```
List<String> frutas = List.of("Mamão", "Abacaxi", "Laranja");
```

```
for (String fruta : frutas) {  
    System.out.println(fruta);  
}
```

Funciona igualmente com arrays:

```
int[] numeros = {10, 20, 30};  
  
for (int numero : numeros) {  
    System.out.println(numero);  
}
```

Prefira o for-each sempre que não precisar da posição. Além de mais legível, ele elimina completamente o risco de acessar um índice inválido.

O for aninhado

Um for dentro do outro. É a base para tabelas, matrizes e desenhos:

```
for (int linha = 1; linha <= 3; linha++) {  
    for (int coluna = 1; coluna <= 3; coluna++) {  
        System.out.print((linha * coluna) + "\t");  
    }  
    System.out.println();  
}
```

Saída:

```
1  2  3  
2  4  6  
3  6  9
```

Entenda o mecanismo: para **cada** volta do laço externo, o laço interno executa **todas** as suas voltas. Aqui são 3 voltas externas x 3 internas = 9 execuções da linha do meio.

O `System.out.println()` vazio, depois do laço interno, é o que quebra a linha ao final de cada linha da tabela.

Vamos analisar o código

Crie o arquivo `Notas.java`:

```
import java.util.ArrayList;  
import java.util.List;  
import java.util.Scanner;  
  
public class Notas {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        List<Double> notas = new ArrayList<>();  
  
        System.out.println("Quantas notas você quer cadastrar?");
```

```

1      int quantidade = Integer.parseInt(scanner.nextLine());      //

2      for (int i = 1; i <= quantidade; i++) {                      //

        System.out.println("Digite a nota " + i + ":");
        notas.add(Double.parseDouble(scanner.nextLine()));
    }

    double soma = 0;
    for (double nota : notas) {                                    //

3        soma += nota;
    }

    System.out.println("--- NOTAS CADASTRADAS ---");
    for (int i = 0; i < notas.size(); i++) {                      //

4        System.out.printf("%dª nota: %.2f%n", (i + 1),
notas.get(i));
    }

    double media = soma / notas.size();                          //

5    System.out.printf("Média: %.2f%n", media);

    int acimaDaMedia = 0;
    for (double nota : notas) {                                    //

6        if (nota > media) {
            acimaDaMedia++;
        }
    }

    System.out.println("Notas acima da média: " + acimaDaMedia);

    scanner.close();
}

```

Linha 1: lê quantas notas serão cadastradas. É esse valor que define o número de repetições — por isso o `for` é o comando adequado aqui.

Linha 2: repete a leitura a quantidade de vezes informada. Note que iniciei o contador em 1 e usei `<= quantidade`, porque aqui o número serve para o usuário ler (“Digite a nota 1”), não como índice.

Linha 3: percorre as notas com o `for-each` para somar todas. Como não preciso da posição, essa é a forma mais limpa.

Linha 4: aqui eu **preciso** da posição, para apresentar a numeração. Por isso voltei ao `for` tradicional. Repare na condição `i < notas.size()`, com o sinal de menor, e no índice começando em zero.

Linha 5: calcula a média dividindo a soma pela quantidade de elementos.

Linha 6: percorre as notas novamente, agora contando quantas estão acima da média. Não há problema algum em percorrer a mesma lista várias vezes.

De olho nos detalhes

O erro de um a mais

```
List<String> nomes = List.of("Ana", "Bruno", "Carla");

for (int i = 0; i <= nomes.size(); i++) {    // ERRADO: <=
    System.out.println(nomes.get(i));        // quebra na quarta volta
}
```

A lista tem 3 elementos e índices 0, 1 e 2. Com `<=`, o laço tenta acessar o índice 3 e o programa é encerrado com `IndexOutOfBoundsException`.

Esse erro é tão comum que ganhou nome próprio na literatura: *off-by-one*, o erro de um a mais. A forma correta é sempre:

```
for (int i = 0; i < lista.size(); i++)
```

Comece em 0 e use o sinal de **menor**. Sem exceção.

Não altere a lista enquanto a percorre

```
for (String nome : nomes) {
    if (nome.startsWith("A")) {
        nomes.remove(nome);    // ConcurrentModificationException
    }
}
```

Remover elementos de uma lista durante um `for-each` provoca erro. Quando precisar fazer isso, use o `for` tradicional percorrendo de trás para frente, ou construa uma lista nova apenas com os elementos que deseja manter.

Contando de trás para frente

O `for` não precisa sempre crescer:

```
for (int i = 10; i >= 1; i--) {
    System.out.println(i);
}
```

E também não precisa avançar de um em um:

```
for (int i = 0; i <= 20; i += 2) {    // apenas os pares
    System.out.println(i);
}
```

Exercícios resolvidos

1) Crie um programa que apresente a tabuada do 7.

```
public class Exercicio01 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            System.out.println("7 x " + i + " = " + (7 * i));  
        }  
    }  
}
```

Compare com o exercício 2 da lição 13, feito com `while`. Mesmo resultado, uma linha a menos e a lógica do laço toda visível em um só lugar.

2) Crie um programa que desenhe um triângulo de asteriscos com 5 linhas.

```
public class Exercicio02 {  
    public static void main(String[] args) {  
        for (int linha = 1; linha <= 5; linha++) {  
            for (int coluna = 1; coluna <= linha; coluna++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

Saída:

```
*  
**  
***  
****  
*****
```

O segredo está na condição do laço interno: `coluna <= linha`. Como `linha` cresce a cada volta externa, o laço interno executa mais vezes a cada linha — e é isso que forma o triângulo.

3) Crie um programa que leia 5 números e apresente quantos são positivos, negativos e zero.

```
import java.util.Scanner;  
  
public class Exercicio03 {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        int positivos = 0;  
        int negativos = 0;  
        int zeros = 0;
```

```

for (int i = 1; i <= 5; i++) {
    System.out.println("Digite o número " + i + ":");
    int numero = Integer.parseInt(scanner.nextLine());

    if (numero > 0) {
        positivos++;
    } else if (numero < 0) {
        negativos++;
    } else {
        zeros++;
    }
}

System.out.println("Positivos: " + positivos);
System.out.println("Negativos: " + negativos);
System.out.println("Zeros: " + zeros);

scanner.close();
}
}

```

Três contadores independentes, todos iniciados em zero antes do laço. Um padrão que se repete muito.

4) Crie um programa que apresente os 10 primeiros números da sequência de Fibonacci.

```

public class Exercicio04 {
    public static void main(String[] args) {
        int anterior = 0;
        int atual = 1;

        for (int i = 1; i <= 10; i++) {
            System.out.print(anterior + " ");

            int proximo = anterior + atual;
            anterior = atual;
            atual = proximo;
        }

        System.out.println();
    }
}

```

Saída: 0 1 1 2 3 5 8 13 21 34

Repare no uso da variável auxiliar proximo. Sem ela, a atribuição a anterior destruiria o valor necessário para calcular o passo seguinte — a mesma ideia do exercício de trocar valores entre variáveis, da lição 6.

5) Crie um programa que leia uma palavra e apresente cada letra em uma linha.

```

import java.util.Scanner;

public class Exercicio05 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite uma palavra:");
        String palavra = scanner.nextLine().strip();

        for (int i = 0; i < palavra.length(); i++) {
            System.out.println((i + 1) + ": " + palavra.charAt(i));
        }

        scanner.close();
    }
}

```

Este exercício une três lições: o for de hoje, o charAt da lição 11 e o índice começando em zero. Repare de novo na condição `i < palavra.length()`.

Exercícios

1. Crie um programa que apresente os números de 1 a 20.
2. Crie um programa que some os números de 1 a 1000 e apresente o total.
3. Crie um programa que apresente os números ímpares de 1 a 50.
4. Crie um programa que leia 5 números e apresente a soma e a média.
5. Crie um programa que percorra uma lista de nomes apresentando cada um com a posição ao lado.
6. Crie um programa que apresente as tabuadas de 1 a 10 usando for aninhado.
7. Crie um programa que desenhe um quadrado de asteriscos com o tamanho informado pelo usuário.
8. Crie um programa que desenhe um triângulo invertido de asteriscos com 5 linhas.
9. Crie um programa que percorra uma lista de preços aplicando 10% de desconto em cada um e apresentando o antes e o depois.
10. Crie um programa que leia 10 números e apresente apenas os pares.
11. Crie um programa que leia um número e apresente todos os seus divisores.
12. Crie um programa que leia um número e informe se ele é primo.
13. Crie um programa que leia uma frase e conte quantas vogais ela tem.
14. Crie um programa que apresente uma contagem regressiva de 10 até 1 e depois a mensagem "Feliz Ano Novo!".
15. Crie um programa que leia 5 nomes, guarde em uma lista e apresente o maior nome (o com mais caracteres).

Dica do dia

Ao percorrer uma lista com índice, sempre $i = 0$ e $i < \text{tamanho}$. Trocar o $<$ por $\leq$ é o erro *off-by-one*, provavelmente o bug mais famoso da programação. E quando você não precisar do índice, use o `for-each`: ele torna esse erro impossível.

DIA 15 — APLICAÇÃO DE CADASTRO COMPLETA

Objetivo do dia

Criar um programa de cadastro completo, organizado em camadas.

Introdução

Nas quatorze lições anteriores você aprendeu todos os comandos necessários para construir um programa: apresentar mensagens, armazenar dados, ler do teclado, calcular, comparar, decidir, criar métodos, manipular textos, trabalhar com listas e repetir comandos.

Na aula de hoje nós vamos juntar tudo isso em uma única aplicação, que cadastra, lista, busca, atualiza e remove clientes.

Mais importante do que o programa em si é a forma como vamos organizá-lo. Até agora, todo o nosso código morava dentro do `main`. A partir de hoje, ele será dividido em partes com responsabilidades diferentes — que é como se constrói software de verdade.

A arquitetura da aplicação

O programa está dividido em cinco arquivos:

Arquivo	Camada	Responsabilidade
<code>Cliente.java</code>	modelo	Representa a informação. Apenas guarda os dados de um cliente.
<code>ServiceClientes.java</code>	dados	Gerencia as informações dos clientes. Representa a camada de acesso a dados.
<code>ControllerClientes.java</code>	controle	Gerencia a comunicação entre a tela e os dados, e aplica as regras.
<code>ViewClientes.java</code>	visualização	Gerencia a comunicação com o usuário e repassa as ações para o controle.
<code>App.java</code>	inicialização	Inicializa a aplicação, montando as peças na ordem correta.

Por que dividir assim?

Porque cada camada pode mudar sozinha.

Se um dia os clientes forem guardados em um banco de dados em vez de uma lista em memória, **apenas o ServiceClientes muda**. Se a aplicação ganhar uma interface gráfica ou virar um site, **apenas a ViewClientes muda**. As regras de negócio continuam intocadas nos dois casos.

Essa ideia é a mesma da lição 10, quando separamos cálculo de apresentação em métodos diferentes. A diferença é a escala: agora estamos separando arquivos inteiros.

Uma regra simples resume tudo:

A camada de dados nunca imprime na tela. A camada de visualização nunca calcula nada.

Pré-requisitos da aula

Crie uma pasta chamada dia15 e coloque os cinco arquivos dentro dela. Nenhuma biblioteca externa é necessária.

Passo 01 — Criar o arquivo Cliente.java

```
/**
 * Modelo de dados: representa um cliente do sistema.
 *
 * Em JavaScript um cliente seria um objeto solto: { id: 1, nome:
 "Ana" }.
 * Em Java criamos uma classe, e com isso ganhamos tipagem,
 autocomplete
 * no editor e erro de compilação quando alguém escreve o nome do
 campo errado.
 */
public class Cliente {

    private int id;
    private String nome;
    private String email;
    private String telefone;

    public Cliente(int id, String nome, String email, String telefone)
    {
        this.id = id;
        this.nome = nome;
        this.email = email;
        this.telefone = telefone;
    }

    public int getId() {
        return id;
    }
}
```

```

    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getTelefone() {
        return telefone;
    }

    public void setTelefone(String telefone) {
        this.telefone = telefone;
    }

    /**
     * Define como o objeto aparece quando é impresso.
     * É o equivalente Java de um método toString() customizado em JS.
     */
    @Override
    public String toString() {
        return String.format("[%d] %-20s %-25s %s", id, nome, email,
            telefone);
    }
}

```

Vamos analisar o código

Esta é a primeira vez que criamos uma classe para representar uma informação, e não para executar um programa. Note que ela não tem main.

Os campos (id, nome, email, telefone) são declarados como private. Isso significa que nenhum código fora desta classe pode alterá-los diretamente.

O construtor é o método com o mesmo nome da classe e sem tipo de retorno. Ele é executado quando usamos `new Cliente(...)` e define os valores iniciais do objeto. A palavra `this` diferencia o campo da classe do parâmetro que chegou com o mesmo nome.

Os **getters e setters** (getNome, setNome, e assim por diante) são o padrão Java para ler e alterar os dados de forma controlada. Repare que o `id` tem apenas *getter*: uma vez atribuído, ele não pode ser alterado — o que faz todo sentido para um identificador.

O **método toString** define como o objeto aparece quando é apresentado com `println`. Sem ele, o Java mostraria algo como `Cliente@6d06d69c`. A anotação `@Override` avisa ao compilador que estamos substituindo um comportamento padrão.

Dentro dele usamos `String.format` com marcadores da lição 11. O `%-20s` significa “texto ocupando 20 caracteres, alinhado à esquerda” — é isso que deixa a listagem em colunas alinhadas.

Passo 02 — Criar o arquivo `ServiceClientes.java`

```
import java.util.ArrayList;
import java.util.List;

/**
 * Camada de dados.
 *
 * Responsabilidade única: guardar e recuperar clientes.
 * Não imprime nada na tela e não lê nada do teclado — se um dia os
 * dados
 * saírem da memória e forem para um banco, só esta classe muda.
 */
public class ServiceClientes {

    private final List<Cliente> clientes = new ArrayList<>();
    private int proximoId = 1;

    /** Localiza a posição de um cliente na lista pelo id. Devolve -1
    se não existir. */
    private int indiceDe(int id) {
        for (int i = 0; i < clientes.size(); i++) {
            if (clientes.get(i).getId() == id) {
                return i;
            }
        }
        return -1;
    }

    /** Devolve uma cópia da lista, para que ninguém de fora altere a
    original. */
    public List<Cliente> buscarTodos() {
        return new ArrayList<>(clientes);
    }

    /** Devolve o cliente com o id informado, ou null se não existir.
    */
    public Cliente buscarPorId(int id) {
        int indice = indiceDe(id);
```

```

        if (indice == -1) {
            return null;
        }
        return clientes.get(indice);
    }

    /** Busca clientes cujo nome contenha o texto informado (sem
diferenciar maiúsculas). */
    public List<Cliente> buscarPorNome(String trecho) {
        List<Cliente> encontrados = new ArrayList<>();
        String alvo = trecho.toLowerCase();

        for (Cliente cliente : clientes) {
            if (cliente.getNome().toLowerCase().contains(alvo)) {
                encontrados.add(cliente);
            }
        }
        return encontrados;
    }

    /** Cria um novo cliente, atribuindo o próximo id automaticamente.
*/
    public Cliente inserir(String nome, String email, String telefone)
    {
        Cliente cliente = new Cliente(proximoId, nome, email,
telefone);
        clientes.add(cliente);
        proximoId++;
        return cliente;
    }

    /** Atualiza os dados de um cliente existente. Devolve false se o
id não existir. */
    public boolean atualizar(int id, String nome, String email, String
telefone) {
        Cliente cliente = buscarPorId(id);
        if (cliente == null) {
            return false;
        }
        cliente.setNome(nome);
        cliente.setEmail(email);
        cliente.setTelefone(telefone);
        return true;
    }

    /** Remove um cliente. Devolve false se o id não existir. */
    public boolean remover(int id) {
        int indice = indiceDe(id);
        if (indice == -1) {
            return false;
        }
    }

```

```

    }
    clientes.remove(indice);
    return true;
}

public int contar() {
    return clientes.size();
}
}

```

Vamos analisar o código

Esta classe tem uma responsabilidade única: guardar e recuperar clientes. Procure por um `System.out.println` nela — não existe nenhum. Essa ausência é proposital.

O **campo `clientes`** é a nossa “base de dados”: uma lista em memória, exatamente como você aprendeu na lição 12.

O **campo `proximoId`** garante que cada cliente receba um identificador único. Ele é incrementado a cada inserção, com o `++` da lição 6.

O **método `indiceDe`** é privado, ou seja, só pode ser usado dentro desta classe. Ele percorre a lista procurando o cliente pelo id e devolve a posição, ou `-1` quando não encontra. É o `for` da lição 14 combinado com o `if` da lição 8.

O **método `buscarTodos`** devolve `new ArrayList<>(clientes)`, ou seja, **uma cópia da lista**. Isso impede que código externo altere os dados originais por acidente. É um cuidado pequeno que evita bugs difíceis de encontrar.

O **método `buscarPorNome`** usa `toLowerCase` e `contains`, da lição 11, para fazer uma busca que não diferencia maiúsculas de minúsculas.

Os **métodos `atualizar` e `remove`** devolvem `boolean` informando se a operação deu certo. Quem chamou decide o que fazer com essa informação — a decisão não é desta camada.

Passo 03 — Criar o arquivo `ControllerClientes.java`

```

import java.util.List;

/**
 * Camada de controle.
 *
 * Faz a ponte entre a tela e os dados: recebe o que o usuário
 * digitou,
 * valida, decide o que fazer e devolve uma mensagem pronta para
 * exibição.
 * É aqui que ficam as regras de negócio.
 */
public class ControllerClientes {

    private final ServiceClientes service;

```

```

    public ControllerClientes(ServiceClientes service) {
        this.service = service;
    }

    public String cadastrar(String nome, String email, String
telefone) {
        if (nome == null || nome.isBlank()) {
            return "Erro: o nome é obrigatório.";
        }
        if (!email.contains("@")) {
            return "Erro: e-mail inválido.";
        }

        Cliente cliente = service.inserir(nome.strip(), email.strip(),
telefone.strip());
        return "Cliente cadastrado com sucesso. Id: " +
cliente.getId();
    }

    public String listar() {
        List<Cliente> clientes = service.buscarTodos();

        if (clientes.isEmpty()) {
            return "Nenhum cliente cadastrado.";
        }

        StringBuilder saida = new StringBuilder();
        saida.append("--- CLIENTES CADASTRADOS
").append(clientes.size()).append(" ---\n");
        for (Cliente cliente : clientes) {
            saida.append(cliente).append("\n");
        }
        return saida.toString().strip();
    }

    public String buscarPorNome(String trecho) {
        if (trecho == null || trecho.isBlank()) {
            return "Erro: informe um texto para a busca.";
        }

        List<Cliente> encontrados =
service.buscarPorNome(trecho.strip());
        if (encontrados.isEmpty()) {
            return "Nenhum cliente encontrado com o nome informado.";
        }

        StringBuilder saida = new StringBuilder("--- RESULTADO DA
BUSCA ---\n");
        for (Cliente cliente : encontrados) {
            saida.append(cliente).append("\n");
        }
    }

```

```

    }
    return saida.toString().strip();
}

public String atualizar(int id, String nome, String email, String
telefone) {
    if (service.buscarPorId(id) == null) {
        return "Erro: não existe cliente com o id " + id + ".";
    }
    if (nome.isBlank()) {
        return "Erro: o nome é obrigatório.";
    }

    service.atualizar(id, nome.strip(), email.strip(),
telefone.strip());
    return "Cliente atualizado com sucesso.";
}

public String remover(int id) {
    boolean removido = service.remover(id);
    if (!removido) {
        return "Erro: não existe cliente com o id " + id + ".";
    }
    return "Cliente removido com sucesso.";
}

public Cliente buscarPorId(int id) {
    return service.buscarPorId(id);
}
}

```

Vamos analisar o código

Aqui moram as regras de negócio: nome obrigatório, e-mail precisa conter @, id precisa existir.

O construtor recebe o service. Repare que o controller não cria o ServiceClientes — ele o recebe pronto. Isso se chama injeção de dependência, e é a base de praticamente todo framework Java moderno. Você vai reencontrar esse padrão o resto da vida.

Cada método devolve uma mensagem de texto em vez de imprimi-la. Assim, quem decide como apresentar é a camada de visualização. Se amanhã essa mensagem precisar aparecer em uma tela gráfica ou em uma página web, o controller não muda nada.

O StringBuilder aparece na listagem porque estamos montando um texto dentro de um laço. Como String é imutável (lição 11), concatenar com + dentro de uma repetição cria um objeto novo a cada volta. Para poucas linhas isso não faz diferença prática, mas é um bom hábito desde o começo.

As validações usam isBlank em vez de isEmpty, porque um nome com apenas espaços também deve ser rejeitado.

Passo 04 — Criar o arquivo ViewClientes.java

```
import java.util.Scanner;

/**
 * Camada de visualização.
 *
 * Responsabilidade única: conversar com o usuário. Lê do teclado,
 * imprime na tela e repassa as ações para o controller.
 * Nenhuma regra de negócio mora aqui.
 */
public class ViewClientes {

    private final ControllerClientes controller;
    private final Scanner scanner;

    public ViewClientes(ControllerClientes controller, Scanner
scanner) {
        this.controller = controller;
        this.scanner = scanner;
    }

    public void iniciar() {
        int opcao;

        do {
            apresentarMenu();
            opcao = lerOpcao();

            switch (opcao) {
                case 1 -> cadastrar();
                case 2 -> System.out.println(controller.listar());
                case 3 -> buscarPorNome();
                case 4 -> atualizar();
                case 5 -> remover();
                case 0 -> System.out.println("Encerrando o sistema.
Até logo!");
                default -> System.out.println("Opção inválida. Tente
novamente.");
            }

            System.out.println();
        } while (opcao != 0);
    }

    private void apresentarMenu() {
        System.out.println("""
=====
                CADASTRO DE CLIENTES
=====
""");
    }
}
```

```

        1 - Cadastrar cliente
        2 - Listar clientes
        3 - Buscar por nome
        4 - Atualizar cliente
        5 - Remover cliente
        0 - Sair
        =====");
    System.out.print("Escolha uma opção: ");
}

/** Lê a opção do menu sem quebrar o programa quando o usuário
digita texto. */
private int lerOpcao() {
    String entrada = scanner.nextLine().strip();
    try {
        return Integer.parseInt(entrada);
    } catch (NumberFormatException e) {
        return -1;
    }
}

private void cadastrar() {
    System.out.print("Nome: ");
    String nome = scanner.nextLine();

    System.out.print("E-mail: ");
    String email = scanner.nextLine();

    System.out.print("Telefone: ");
    String telefone = scanner.nextLine();

    System.out.println(controller.cadastrar(nome, email,
telefone));
}

private void buscarPorNome() {
    System.out.print("Digite parte do nome: ");
    String trecho = scanner.nextLine();
    System.out.println(controller.buscarPorNome(trecho));
}

private void atualizar() {
    System.out.println(controller.listar());
    System.out.print("Id do cliente que deseja atualizar: ");
    int id = lerOpcao();

    Cliente cliente = controller.buscarPorId(id);
    if (cliente == null) {
        System.out.println("Erro: não existe cliente com o id " +
id + ".");
    }
}

```

```

        return;
    }

    System.out.print("Novo nome (" + cliente.getNome() + "): ");
    String nome = scanner.nextLine();

    System.out.print("Novo e-mail (" + cliente.getEmail() + "): ");
    String email = scanner.nextLine();

    System.out.print("Novo telefone (" + cliente.getTelefone() + "): ");
    String telefone = scanner.nextLine();

    System.out.println(controller.atualizar(id, nome, email, telefone));
}

private void remover() {
    System.out.println(controller.listar());
    System.out.print("Id do cliente que deseja remover: ");
    int id = lerOpcao();
    System.out.println(controller.remover(id));
}
}

```

Vamos analisar o código

Esta classe é a costura de quase tudo que você aprendeu:

- o **do/while** da lição 13 mantém o menu na tela até o usuário escolher sair;
- o **switch com seta** da lição 09 direciona cada opção;
- o **text block** da lição 11 desenha o menu sem precisar de vários `println`;
- o **Scanner** da lição 05 lê as escolhas;
- os **métodos privados curtos** da lição 10 separam cada ação.

O método **lerOpcao** traz o único conceito realmente novo do livro: o `try/catch`. Ele impede que o programa seja encerrado quando o usuário digita uma letra onde se esperava um número. `Integer.parseInt("abc")` provoca um erro chamado `NumberFormatException`; o `catch` intercepta esse erro e devolve `-1`, que cai no `default` do `switch` e apenas apresenta “opção inválida”.

Você não precisa dominar tratamento de exceções agora. Guarde apenas a ideia: o `try` protege um trecho que pode falhar, e o `catch` decide o que fazer quando falha.

O método **atualizar** apresenta o valor atual entre parênteses em cada pergunta. É um detalhe pequeno de experiência do usuário que faz muita diferença na prática.

Passo 05 — Criar o arquivo App.java

```
import java.util.Scanner;

/**
 * Ponto de entrada da aplicação.
 *
 * Responsabilidade única: montar as peças na ordem certa e iniciar a
 * tela.
 * Repare que o Scanner é criado uma única vez aqui e emprestado para
 * a view.
 */
public class App {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        ServiceClientes service = new ServiceClientes();
        ControllerClientes controller = new
ControllerClientes(service);
        ViewClientes view = new ViewClientes(controller, scanner);

        view.iniciar();

        scanner.close();
    }
}
```

Vamos analisar o código

Três linhas montam a aplicação inteira, e a ordem importa: o service é criado primeiro, entregue ao controller, que por sua vez é entregue à view.

Repare também que o Scanner é criado **uma única vez** aqui e emprestado para a view. Criar vários Scanner sobre System.in no mesmo programa provoca comportamentos estranhos de leitura — é um erro comum e difícil de diagnosticar.

Como executar a aplicação

Com os cinco arquivos na mesma pasta:

```
javac *.java
java App
```

Se os acentos aparecerem como ? no terminal, force a codificação:

```
java -Dstdout.encoding=UTF-8 App
```

No Windows, executar chcp 65001 antes do programa também resolve.

De olho nos detalhes

O que significa “camada”

Uma camada é um conjunto de classes com uma mesma responsabilidade, que conversa apenas com as camadas vizinhas. No nosso desenho:

View → Controller → Service

A view nunca fala diretamente com o service. O service nunca sabe que existe uma tela. Essa disciplina é o que mantém um sistema compreensível quando ele cresce.

Por que o Cliente não tem regras

O Cliente apenas guarda dados. As validações ficam no controller. Existem abordagens diferentes em projetos reais, algumas colocando parte das regras no próprio modelo — mas, para começar, manter o modelo simples é o caminho mais claro.

Isto já é orientação a objetos

Você acabou de criar classes, construtores, objetos, campos privados e métodos públicos. Na prática, você deu os primeiros passos em orientação a objetos sem que fosse necessário nenhum capítulo teórico sobre o assunto.

O que falta estudar depois — herança, interfaces, polimorfismo — vai fazer muito mais sentido agora que você tem um sistema funcionando nas mãos.

Exercícios

Estes exercícios transformam a aplicação em um projeto de portfólio. Faça um de cada vez e teste antes de passar ao próximo.

1. Adicione um campo `cidade` ao cliente, propagando a mudança pelas cinco camadas.
2. Impeça o cadastro de dois clientes com o mesmo e-mail.
3. Valide o telefone: aceite apenas dígitos, com 10 ou 11 caracteres.
4. Adicione a opção “6 - Ordenar por nome” ao menu.
5. Adicione a opção “7 - Total de clientes cadastrados”.
6. Crie uma busca por e-mail, além da busca por nome.
7. Peça confirmação (“s” ou “n”) antes de remover um cliente.
8. Faça a listagem apresentar um cabeçalho de colunas alinhado com `printf`.
9. Adicione um campo `ativo` do tipo `boolean` e faça a remoção apenas desativar o cliente. Depois filtre a listagem para mostrar somente os ativos.
10. Permita que o usuário pressione Enter para manter o valor atual durante a atualização.
11. Crie uma segunda entidade (por exemplo, `Produto`) com as suas próprias camadas, e adicione ao menu principal a escolha entre gerenciar clientes ou produtos.
12. Salve os clientes em um arquivo de texto ao sair e carregue-os ao iniciar. Pesquise a classe `java.nio.file.Files`.

O exercício 12 é o mais difícil do livro, e o mais recompensador: ele transforma o seu programa em algo que não perde os dados ao ser fechado.

Dica do dia

Cada arquivo do seu programa deve ter uma responsabilidade que você consiga explicar em uma frase. Se você precisar de “e” para descrever o que uma classe faz, provavelmente ela deveria ser duas.

ENCERRAMENTO

Parabéns

Se você chegou até aqui resolvendo os exercícios, você não “leu um livro sobre Java”. Você escreveu mais de duzentos programas.

Faça um exercício de memória: volte ao Dia 03 e olhe o primeiro programa, aquele de três linhas que apresentava uma mensagem na tela. Agora olhe a aplicação que você acabou de construir. É a mesma pessoa, quinze lições depois.

Os erros mais comuns e o que eles significam

Guarde esta tabela. Ela vai te economizar muitas buscas na internet.

Mensagem	Causa	Solução
<code>';' expected</code>	falta um ponto e vírgula	acrescente o ;
<code>cannot find symbol</code>	nome escrito errado ou falta um import	confira a grafia e os imports
<code>class X is public, should be declared in a file named X.java</code>	nome do arquivo diferente do nome da classe	renomeie um dos dois
<code>incompatible types</code>	tipo errado na atribuição	converta com <code>parseInt</code> ou <code>parseDouble</code>
<code>variable x might not have been initialized</code>	uso da variável antes de atribuir valor	inicialize na declaração
<code>missing return statement</code>	existe um caminho no método sem return	garanta retorno em todos os ramos
<code>non-static method cannot be referenced from a static context</code>	método de instância chamado a partir do main	marque o método como <code>static</code>
<code>NullPointerException</code>	método chamado sobre uma variável null	verifique <code>!= null</code> antes
<code>IndexOutOfBoundsException</code>	índice fora do intervalo da lista	use <code>i < lista.size()</code>

Mensagem	Causa	Solução
NumberFormatException	texto não numérico no <code>parseInt</code>	valide a entrada ou use <code>try/catch</code>
ConcurrentModificationException	lista alterada durante um <code>for-each</code>	percorra com <code>for</code> tradicional ou monte uma nova lista

As dez armadilhas do Java para iniciantes

1. `10 / 3` resulta em 3, e não em 3.33.
2. Textos se comparam com `.equals()`, nunca com `==`.
3. Não existe valor “verdadeiro por aproximação”: condições precisam ser `boolean`.
4. `length()` é método em `String`, mas `length` é propriedade em array.
5. Não existe `undefined`, e variáveis não inicializadas não podem ser usadas.
6. Array tem tamanho fixo; para lista dinâmica, use `ArrayList`.
7. Textos são imutáveis: sempre guarde o valor devolvido pelos métodos.
8. Índices começam em zero e terminam em `tamanho - 1`.
9. O `switch` clássico precisa de `break`; prefira a forma com seta.
10. Tudo vive dentro de uma classe, e o programa começa no `main`.

O que estudar depois

Você concluiu o núcleo da programação. Estes são os próximos passos naturais, nesta ordem:

- 1. Orientação a objetos.** Herança, interfaces, polimorfismo, encapsulamento e classes abstratas. É onde Java realmente brilha, e a aplicação da lição 15 vai fazer muito mais sentido depois disso.
- 2. Collections.** Map, Set, Queue e quando usar cada uma. O `ArrayList` é apenas a porta de entrada.
- 3. Tratamento de exceções.** `try`, `catch`, `finally` e a criação de exceções próprias.
- 4. Streams e lambdas.** A forma moderna de processar coleções em Java, com `filter`, `map` e `collect`.
- 5. Maven ou Gradle.** As ferramentas que gerenciam dependências e a construção de projetos Java.
- 6. JDBC e depois JPA/Hibernate.** Para trocar a lista em memória por um banco de dados real.
- 7. Spring Boot.** O framework que domina o mercado Java hoje, e o destino da maioria de quem aprende a linguagem profissionalmente.

Uma sugestão prática antes de correr para o Spring: refaça a aplicação da lição 15 duas vezes. A primeira usando herança e interfaces; a segunda com persistência em arquivo. Reescrever o mesmo problema com ferramentas melhores ensina mais do que começar um projeto novo do zero.

Uma última palavra

Todo programador que você admira já ficou travado em um erro de compilação por horas. Todos eles já escreveram um laço infinito. Todos já compararam textos com `==` e perderam uma tarde inteira por causa disso.

A diferença entre quem desiste e quem continua não é talento: é a disposição de encarar o próximo erro como informação, e não como veredito.

Continue escrevendo código.

Bons estudos!